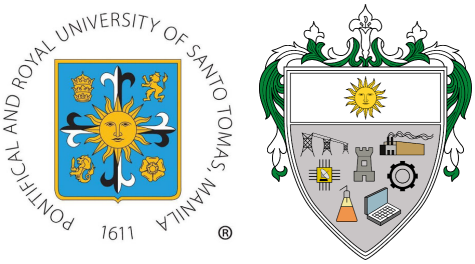

An Evaluation of Multi-Agent Reinforcement Learning for Dynamic Bus Scheduling Under Non-Ideal Conditions

A thesis submitted in partial fulfilment

of the requirements for the degree of

Bachelor of Science in Electronics Engineering

Department of Electronics Engineering

Faculty of Engineering

University of Santo Tomas

Proponents:

Badal, Khalil T.

Lopez, Elijah David B.

Mananguit, Jared Asher N.

Marquez, Rence Joseph L.

Medenilla, Jose Anton M.

Research Supervisor:

Asst. Prof. Kanny Krizzy D. Serrano, MSc

Date: August 26, 2026

Word Count: 15398

AI Use Disclosure

The authors acknowledge the use of artificial intelligence (AI)-assisted tools during the preparation of this manuscript. ChatGPT and Grammarly were used primarily as editorial, organizational, and formatting support tools during the writing and manuscript-preparation process.

Grammarly was used throughout all chapters for grammar checking, spelling correction, punctuation review, and sentence-level clarity suggestions. ChatGPT was used selectively for proofreading and language refinement of draft prose to improve readability, reduce informal phrasing, and smooth transitions between sections. It was also used to provide organizational feedback on manuscript structure, particularly in refining the funnel progression of the Review of Related Work, summarizing disturbance coverage across studies already collected by the authors, checking consistency of terminology, notation, section numbering, and symbol usage across Chapters 1–3, and identifying undefined terms, missing bibliography entries, and citation inconsistencies prior to compilation.

In addition, AI-assisted tools were used for document-preparation and Latex-related support tasks, including table formatting, float placement adjustments, bibliography troubleshooting, citation-key checking, and resolving compilation issues encountered during Overleaf manuscript assembly.

AI tools were employed strictly as editorial, organizational, and formatting assistants during the preparation of this manuscript. They were not used to generate research findings, conduct simulations, or produce technical conclusions. All core research decisions,including the proposed simulation framework, methodological direction, disturbance-modeling approach, evaluation design, and interpretation of the reviewed literature, were determined and validated by the authors. All AI-generated suggestions were reviewed, revised where necessary, and approved prior to inclusion. The authors assume full responsibility for the accuracy, originality, and integrity of the proposed work.

Contents

List of Figures

List of Tables

Chapter 1

Introduction and Literature Review

1.1 Background of the Study

Public transportation has long been an essential part of urban mobility, taking different forms and operating across different terrains, with routes generally determined by geographical conditions and commuter needs. As urbanization has increased over time, so has the demand for transportation, with growing urban populations relying increasingly on public transit—and high-frequency bus service in particular—to move through congested cities [1]. As the number of motor vehicles competing for the same road space continues to rise, traffic congestion correspondingly worsens, increasing the importance of reliable transportation for commuters seeking to reach their destinations.

These transportation challenges are particularly evident in the Philippines. Despite a predicted decline in urbanization rates across Southeast Asia, the Philippines is experiencing the opposite trend [2], concentrating growing transportation demand onto major corridors such as the EDSA Carousel in Metro Manila. In Metro Manila, severe traffic congestion, stemming from insufficient road infrastructure and poor traffic mitigation policies, has resulted in a highly inefficient public transport system [3]. As a response, the Philippine government launched the EDSA Bus Carousel in 2020. It is the only busway system in Metro Manila implemented by the government to have a designated lane, separated from other cars by concrete barriers [4]. Throughout the years of implementation, a large number of commuters have utilized the system, with an average of 183,000 daily passengers, climbing up to 320,000 during peak periods, and a ridership of nearly 67 million in 2025, up from 63.02 million in 2024 (Figure 1.1) [5]. With that amount of ridership, the EDSA

Bus Carousel can be safely assumed to be one of the backbones of Metro Manila commuting.

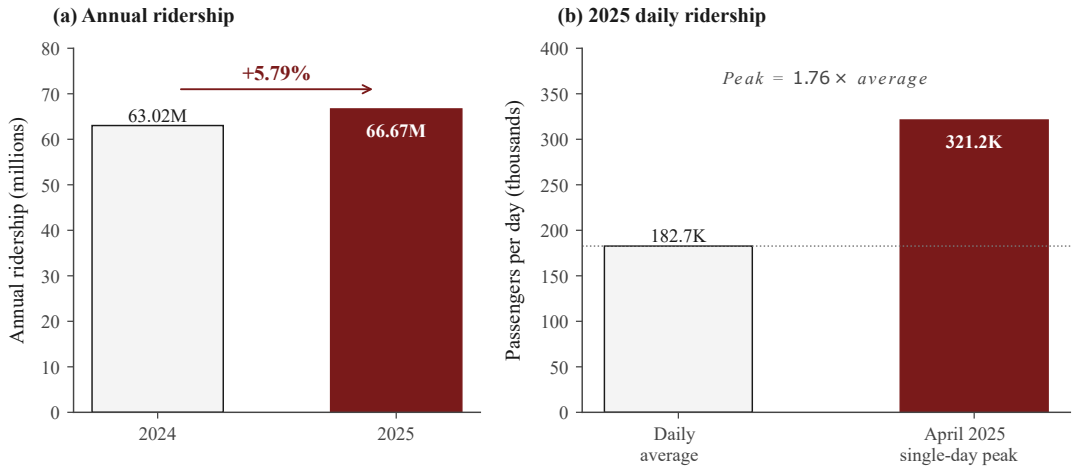

Figure 1.1: EDSA Carousel ridership. (a) Annual ridership grew 5.79% from 63.02M in 2024 to 66.67M in 2025. (b) Daily ridership in 2025 averaged 182,655 passengers, with a single-day peak of 321,186 in April 2025 — approximately 1.76× the daily mean [5].

Despite the gradual increase in demand, the EDSA Bus Carousel still faces significant operational issues that affect the quality of bus routing. Firstly, the LTFRB relied on a tedious manual monitoring process by deploying personnel at each stop, creating decisions through empirical observation [6]. In relation to that, it is also clear that bus bunching has not been resolved; most of the time, the designated dispatch schedule is not followed [7]. There is an acknowledged lack of a structured monitoring framework for operations (e.g., dispatching, dwell time, bus bunching), in which adjustments are often made only after recurring issues surface [8]. Moreover, mechanical failures and bus breakdowns are also prominent in Metro Manila, highlighting that poor vehicle condition and the constant threat of mid-route breakdowns are significant chronic stressors for Filipino bus drivers [4]. Furthermore, environmental stochasticity severely destabilizes operations: empirical studies on Philippine expressways show that increasing rainfall intensity significantly reduces average traffic speed and free-flow capacity [9].

As Figure 1.2 shows, average speeds are reduced by about 5.34% under light rain, 6.3% under moderate rain, and 7.4% under heavy rain; average volume decreased by 2.92%, 10.62%, and 12.39%, and capacity was reduced by 3.67%, 7.6%, and 17.44% under light, moderate, and heavy rain, respectively [9]. Severe weather events often initiate localized flooding at key segments or force operational halts and emergency speed restrictions that further degrade busway performance [10–12]. Given these operational hurdles, the bus dispatching process of the EDSA Carousel can be modeled as a Vehicle Scheduling Problem (VSP).

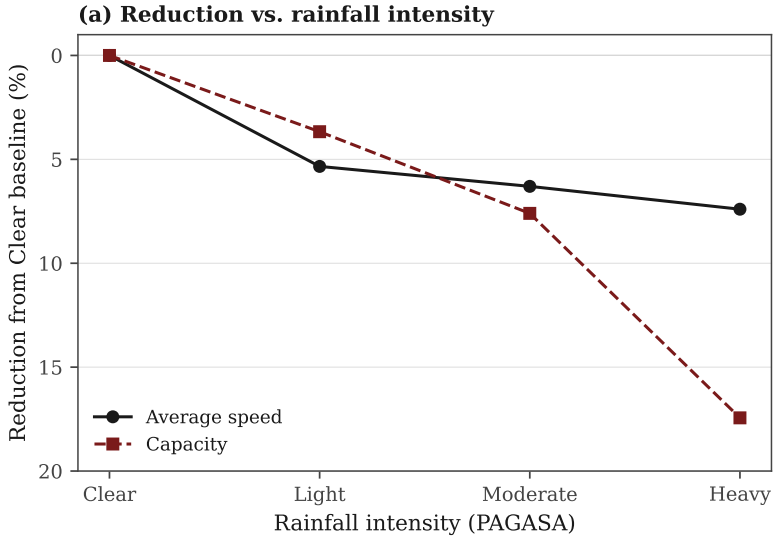

Figure 1.2: Reduction in average speed and free-flow capacity relative to the clear-weather baseline across PAGASA rainfall categories. Capacity loss grows sharply at heavy intensities (−17.4%), whereas mean speed degrades more moderately (−7.4%), adapted from [9].

The same considerations apply beyond Metro Manila. Comparable bus systems elsewhere in the Global South face similar challenges: public transport in Latin American cities has undergone a comparable shift in how it is provided and relied upon [13]. In Montería, Colombia, for example, a mid-sized bus network of 21 routes has struggled with low service frequency, poor passenger satisfaction, and financial instability under a purely manual, static dispatch rule [14]. These cases show that the operational problem motivating this study—reliable bus service under real-world disturbances—recurs across corridors worldwide, underscoring the need for scheduling approaches that generalize beyond any single network's local conditions.

Traditionally, the VSP has been addressed through static scheduling, which uses fixed timetables and headways derived from historical averages [15]. However, these rigid, rule-based methods often perform poorly in stochastic environments because they cannot adapt to real-time disruptions, motivating a shift toward more dynamic scheduling frameworks [7]. This shift also motivated the adoption of data-driven methodologies. Initial studies applied traditional machine learning (ML) to demand forecasting and traffic prediction [16], but these models are primarily limited to predictive tasks and lack the active control mechanisms required for real-time decision-making in a closed-loop system. Subsequent studies introduced Single-Agent Reinforcement Learning (SARL), which enabled autonomous control, but these frameworks frequently suffer from the "curse of dimensionality" as the joint state and action spaces grow with the number of vehicles [17]. To address these limitations, Multi-Agent Reinforcement Learning (MARL) has emerged as a promising approach, offering decentralized execution that enables scalable, collaborative con-

trol among multiple buses to address issues such as bus bunching [18].

Although recent studies have demonstrated the potential of MARL-based dynamic scheduling to improve transit adaptability and overall reliability, a critical research gap remains. Existing MARL bus-scheduling models are primarily evaluated in controlled simulation settings and are typically tested against isolated disruptions. No prior work has evaluated these models under combined, simultaneous severe disruptions—heavy rain, mechanical breakdowns, traffic variations, and sudden passenger surges—representative of real-world corridors such as EDSA and its international counterparts. This study therefore evaluates a MARL-based dynamic bus scheduling model under these non-ideal, stochastic conditions, using a public-data-calibrated empirical case selected for the data provenance, integrity, and stop-level granularity required for rigorous evaluation.

Because the EDSA operational record were not be able to be obtained, the study is demonstrated on CapMetro Rapid Route 801 in Austin, Texas—a high-frequency arterial corridor whose public data is complete enough to calibrate and audit, and whose disturbance profile mirrors the motivating corridors above. CapMetro identifies Route 801 as its Rapid North Lamar/South Congress service [19]; however, current schedule information is not used to assign historical direction labels. The empirical source is CapMetro's public Automatic Passenger Counter (APC) dataset for July–December 2021 [20]. The portal exposes 47 text-typed fields, including service and event timestamps, route and direction codes, stop IDs, boardings, alightings, onboard load, dwell time, revenue travel time and distance, event coordinates, and data-quality indicators.

The case-route decision was made before controller evaluation using identical filters for Routes 801 and 803: the reported route had to match the active route, both import-error fields had to equal zero, the stop ID could not equal zero, and the direction code had to be 4 or 6. The reproduced comparison in Table 1.1 supports Route 801 because it supplies more clean stop events, recorded boardings, and positive-time/positive-distance segments while retaining comparable GPS quality. This is a data-coverage justification, not a claim that Route 801 provides better or worse service.

Table 1.1: Reproduced public-data coverage used to select the primary CapMetro case route.

Metric	**Route 801**	**Route 803**
All route records	547,616	468,689
Clean stop events, direction codes 4 and 6	455,654	376,801
Recorded boardings	810,309	532,063
Positive-time/positive-distance segments	441,779	361,852
High-quality GPS records	98.136%	97.479%

The primary one-direction subset uses Route 801 direction code 6 and contains

229,421 clean stop events across 184 service-day codes and 29 distinct stop IDs. The code is intentionally not called northbound or southbound because a checksum-verified 2021 GTFS snapshot has not yet been acquired. The APC file also does not contain passenger arrival timestamps, vehicle capacity, breakdown events, or weather. These absences bound the study design: waiting time and breakdowns are simulation variables, capacity remains a separate source requirement, and weather is joined from an authoritative public source rather than imputed from APC data.

Route 801 is CapMetro's high-frequency MetroRapid service along the North Lamar–South Congress corridor, one of Austin's principal arterial transit spines [19]. The direction-code-6 subset used in this study traverses 29 distinct stops over 184 service-day codes. Consistent with the data-provenance discipline above, these stops are identified by their APC stop ID rather than by name or compass bearing until a checksum-verified 2021 GTFS snapshot is obtained. Figure 1.3 shows the published Route 801 alignment; its eleven numbered markers are schedule *timepoints*[1], while the direction-code-6 subset modelled here comprises the 29 distinct stop IDs served between and including them.

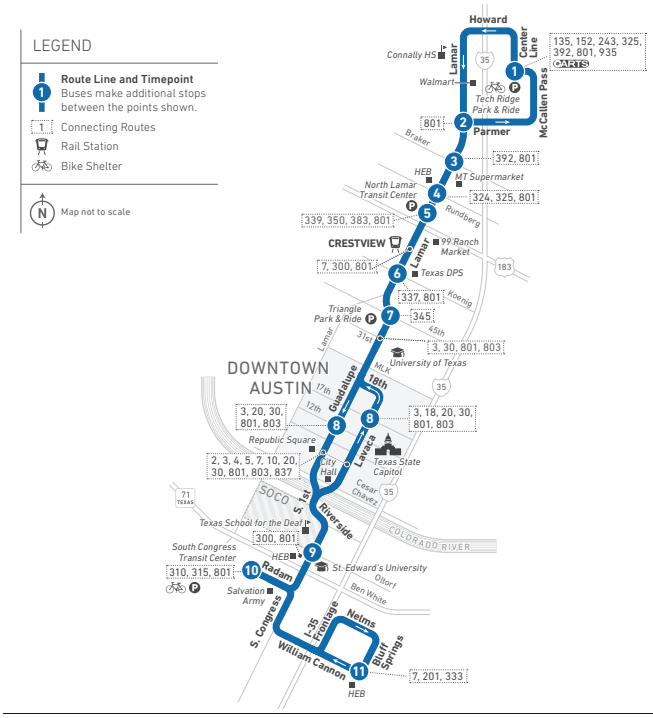

Figure 1.3: CapMetro MetroRapid Route 801 (North Lamar–South Congress). The eleven numbered markers are published schedule *timepoints*; per the operator's legend, buses make additional stops between them, giving the 29 distinct stop IDs in the direction-code-6 subset modelled here. Map source: Capital Metropolitan Transportation Authority [19], reproduced for academic use. The current published alignment is shown; 2021 direction labels and stop names remain gated pending a checksum-verified 2021 GTFS snapshot.

[1]The numbered markers on the published map are schedule timepoints—selected stops a bus may not depart before the scheduled time, used to hold the route on schedule—not the full stop set. Buses serve additional stops between them, so the direction-code-6 subset comprises 29 distinct stop IDs in the APC data [19].

Weather feasibility was tested with NOAA Local Climatological Data Version 2 (LCDv2) [21]. Austin Camp Mabry is the primary station and Austin-Bergstrom is the sensitivity station. After interpreting NOAA observations as local standard time, converting them to `America/Chicago`, and joining the nearest observation within 90 minutes, both stations matched 100% of the 229,421 primary APC events; 11,804 events matched a Camp Mabry rain flag. These coverage results establish that an empirical ordinary-rain exposure can be constructed. They do not establish a causal rain effect: multiplier estimation must control for segment, time of day, and day type. Severe or extreme weather outside the observed support remains an explicitly synthetic stress test.

The remaining research question is not whether the public data contain every disturbance. They do not. It is whether a MARL controller calibrated to observed CapMetro demand, dwell, and segment travel-time behavior remains effective when controlled demand surges, observed ordinary-weather exposure, synthetic out-of-support weather stress, and synthetic vehicle breakdowns are introduced under a declared experimental matrix. The study therefore evaluates robustness without presenting synthetic incidents or passenger waiting times as measured CapMetro facts.

1.2 Review of Related Work

This section surveys prior work on data-driven bus and traffic scheduling using a funnel structure. It starts from the broad transition from static scheduling toward adaptive and learning-based control, narrows to single-agent reinforcement learning and its known scalability limitations in large transit systems, narrows further to Multi-Agent Reinforcement Learning (MARL) and the assumptions commonly used in existing MARL bus-scheduling studies, and finally identifies the robustness gap addressed in this study.

1.2.1 From Static Rules to Machine Learning

Static scheduling refers to the offline determination of bus departure times, headways, fleet allocations, and stop assignments based on historical demand and travel-time averages, fixed in advance of operations and not adjusted in real time in response to observed disturbances [15]. Once deployed, such schedules cannot react to live disturbances such as demand surges, congestion, weather-induced delays, or vehicle breakdowns. The poor performance of fixed timetables under such conditions is well documented: even-headway and schedule-based control degrade rapidly once travel-time and demand variability exceed the narrow band assumed at design time,

allowing small headway perturbations to amplify into bunching [22]. Static schedules therefore remain mathematically inadequate for stochastic traffic environments, where the governing quantities are random variables rather than deterministic constants. Under the specific non-ideal conditions this study targets, the failure modes differ by control strategy. A fixed timetable has no feedback mechanism at all, so once bunching begins nothing in the schedule corrects it. A local reactive rule that holds a bus based only on the gap to the bus ahead can partially correct bunching under ordinary congestion, but has no way to respond to a breakdown, since it observes only the forward gap and not the enlarged gap a failed bus leaves behind it. A more globally aware reactive rule that accounts for both the forward and backward gap improves on this, but still follows a fixed, pre-specified rule rather than a learned response, so it cannot adapt its behavior to the heavier-tailed delays that severe weather introduces. These limitations motivated the transition toward more adaptive and data-driven scheduling methodologies.

Machine Learning (ML) techniques subsequently emerged as important tools for improving operational responsiveness in transportation systems through demand estimation, traffic forecasting, predictive dispatching, and adaptive holding strategies [23, 24]. Unlike static scheduling approaches, ML-assisted frameworks incorporate continuously updated operational information to improve short-term decision support under uncertain traffic conditions [23]. Recent applications to transit operations have demonstrated measurable operational gains. Barrera Hernandez et al. applied passenger-demand forecasting to support a heuristic dispatcher for bus operations in Bogotá, reporting reductions in passenger waiting time and operational cost relative to fixed timetables [14]. Similarly, Usman et al. utilized ML-based density estimation to improve real-time responsiveness in public transit systems [25]. These studies demonstrate the growing role of ML-assisted methodologies in adaptive transit scheduling and operational planning.

Despite these advances, many ML-assisted transit-control frameworks still rely heavily on manually engineered intervention logic and externally designed dispatch heuristics [26]. In several predictive-control and heuristic-dispatch systems, forecasting components improve operational visibility, but the resulting operational responses are still governed by manually specified holding rules, threshold conditions, or predefined intervention strategies [14, 27]. In the framework proposed by Barrera Hernandez et al., for example, a demand forecast feeds a dispatch heuristic that autonomously determines when to release each bus—without a human dispatcher in the loop at runtime—but the decision itself is governed by a fixed, hand-tuned rule based on per-route occupancy and maximum-wait thresholds, calibrated once through grid search, rather than by a policy that adapts its decision logic through experience. Furthermore, the study is limited to terminal dispatch timing and does

not address en-route actions such as holding duration or stop-skipping [14]. Consequently, while ML-assisted scheduling frameworks improve anticipatory planning and operational responsiveness, many existing implementations still rely on forecasts feeding fixed, pre-specified rules rather than a policy that learns and autonomously optimizes its sequential dispatch logic over time.

Table 1.2: Progression from static rules to reinforcement learning along the dimensions that determine real-time control capability.

Paradigm	**Adapts to real-time conditions?**	**Produces a dispatching action?**	**How the action is determined**
Static / heuristic scheduling	No, fixed offline from historical averages	Yes, but actions are predetermined before operation	Pre-set timetables or manually designed heuristic rules that do not adapt to live corridor conditions [15, 22]
Machine-learning assisted scheduling	Yes, forecasts update continuously using observed operational data	Partially; predictions support decisions, but actions are still governed externally	Forecasting models estimate demand or traffic conditions while separate heuristic dispatch rules determine the operational response [14, 25, 27]
Reinforcement learning	Yes, policies continuously condition on live observed system states	Yes, the dispatching action is produced directly by the learned policy	The control policy learns through interaction with the environment to maximize long-term operational performance [28–30]

The decisive distinction is the final column: only reinforcement learning makes the dispatching action itself a learned output. This study instantiates the reinforcement-learning row as a parameter-sharing Double Deep Q-Network under centralized training with decentralized execution (Chapter 3).

1.2.2 Single-Agent Reinforcement Learning and Its Limitations

Reinforcement Learning (RL) closes the prediction–control gap by learning a *policy*: a mapping from observed system states to actions, optimized through trial-and-error interaction with the environment to maximize a long-horizon reward signal [28]. Unlike a forecaster, an RL agent does not just predict what will happen — it decides what to do. The reward signal closes the loop: the agent's action changes the environment, the environment returns a new state and a scalar reward, and the policy is updated so that better actions become more likely the next time a similar state is encountered.

When RL is applied to a multi-vehicle problem such as bus-fleet control, the simplest formulation treats the whole fleet as one entity controlled by a single agent. This is **Single-Agent Reinforcement Learning (SARL)**: one centralized policy observes the full system state and emits a joint action covering every bus at once. SARL has been applied directly to bus holding control, including the STDH-DQN approach of Zhao et al. [31], which uses a self-attention state encoder over

spatial-temporal AVL features, and the more recent SA-DRL approach of Zhang and Zheng [32], which encodes categorical identifiers (vehicle ID, station ID, trip ID) as state features and reports favorable results against MARL baselines on a bidirectional timetabled line. SARL is therefore a real and competitive option in this space; the question is not whether it works but where it begins to strain.

SARL faces three scalability limitations in multi-bus control, each documented in the bus-scheduling and traffic-control literature. These limitations motivate the move to a multi-agent formulation in the next subsection; each has a corresponding design response that MARL provides.

1. **State-space dimensionality growth.** A SARL formulation of N buses on M stops requires a global state $s \in \mathbb{R}^{N \cdot d}$, where d is the per-bus feature count (position, headway, load). The reproduced Route 801 direction-code 6 subset contains $M = 29$ distinct stop IDs; the active fleet size N remains a calibration output rather than an assumed constant. As N grows, the function approximator must also learn approximate permutation invariance over buses: the policy should not depend on the arbitrary index assigned to a vehicle, a property not enforced by default in ordinary neural networks [33].

2. **Joint action-space combinatorial growth.** A SARL controller emits a joint action $a = (a_1, \ldots, a_N) \in A^N$. With the hybrid action space adopted in this study ($|A_i| = 5 \times 2 = 10$, combining a 5-bin holding-strength discretization with a binary skip decision), a SARL controller managing $N = 20$ buses operates over a joint action space of 10^{20} combinations. Although deep RL policies do not enumerate these combinations directly — they sample from a parameterized distribution over actions — the credit-assignment burden remains: a single scalar reward must be attributed across all N simultaneous component actions, and this attribution problem worsens as N grows. MARL with shared policies factorizes the action selection into N independent decisions of size 10, each with its own attributable reward signal.

3. **Sample inefficiency under non-stationarity.** SARL treats the fleet as a single dynamical system; every demand change, breakdown, or weather event triggers re-adaptation of the entire policy. MARL with parameter sharing benefits from *experience aggregation*: every action by every bus generates one training sample for the same shared network, so each bus's experience accelerates learning for all of them [34, 35].

Recent SARL-for-bus-control work that competes successfully against MARL baselines does so partly by hand-engineering categorical agent-identity features into the state representation [32]. This is informative because it suggests that agent identity remains an important representational issue in SARL formulations even at the

high end of reported performance — the dimensionality concern in item (1) above is partially addressed by smuggling identity through input features rather than by a change in policy class. The fix is workable on a fixed corridor but does not transfer cleanly when fleet size or route topology changes.

We note a counter-position. Zhang and Zheng [32] report SA-DRL matching or exceeding MARL baselines on bidirectional timetabled lines, attributing this to data-imbalance issues in MARL when some agents see few episodes. The Route 801 experiment is deliberately restricted to one direction code and a shared agent class, closer to the single-direction assumptions under which MARL has been shown to perform well. This motivates the MARL choice while retaining SARL as a legitimate comparator rather than dismissing it.

To situate the MARL literature reviewed in the next subsection within the broader ML and SARL landscape, Table 1.3 extends the paradigm comparison of Table 1.2 with a disturbance-coverage column, using the same D/S/T/W/B notation as Table 1.4.

Table 1.3: Disturbance coverage across ML and SARL vehicle-scheduling studies and adjacent weather evidence, preceding the MARL-specific comparison in Table 1.4.

Paper	**Paradigm**	**Method**	**Disturbances**
Wang et al. [16]	ML (data-driven)	Bus scheduling model incorporating time-dependent traffic and demand	D
Barrera Hernandez et al. [14]	ML-assisted (heuristic dispatcher)	Passenger-demand forecasting supporting a heuristic dispatcher	D
Zhao et al. [31]	SARL	STDH-DQN; self-attention state encoder over spatial-temporal AVL features	D, T
Zhang and Zheng [32]	SARL	SA-DRL; categorical identity features (vehicle, station, trip ID)	D, T
Sun et al. [36]	ML prediction (non-control)	SVM, KNN, BP, and GA-BP prediction of bus dwell time under measured rainfall	W

D = stochastic demand, S = demand surges, T = traffic-speed perturbation, W = weather exposure or weather-induced travel-time variability, B = discrete vehicle breakdown. Sun et al. provide empirical and predictive weather evidence, not a bus-control or reinforcement-learning baseline.

The comparison separates evidence by function. Sun et al. [36] empirically analyze and predict bus dwell time under measured rainfall, so they support the relevance of W but do not provide a controller baseline. Patil et al. [37] validate a lognormal travel-time form for simulated arterial scenarios, which supplies the distributional form used by the synthetic weather-stress generator (Section 3.2.6), not a bus-control result. The breakdown evidence reviewed below is likewise adjacent rather than prior MARL bus control. Within the reviewed set, no MARL bus-scheduling study evaluates both W and B, which is the narrower gap tested by this study.

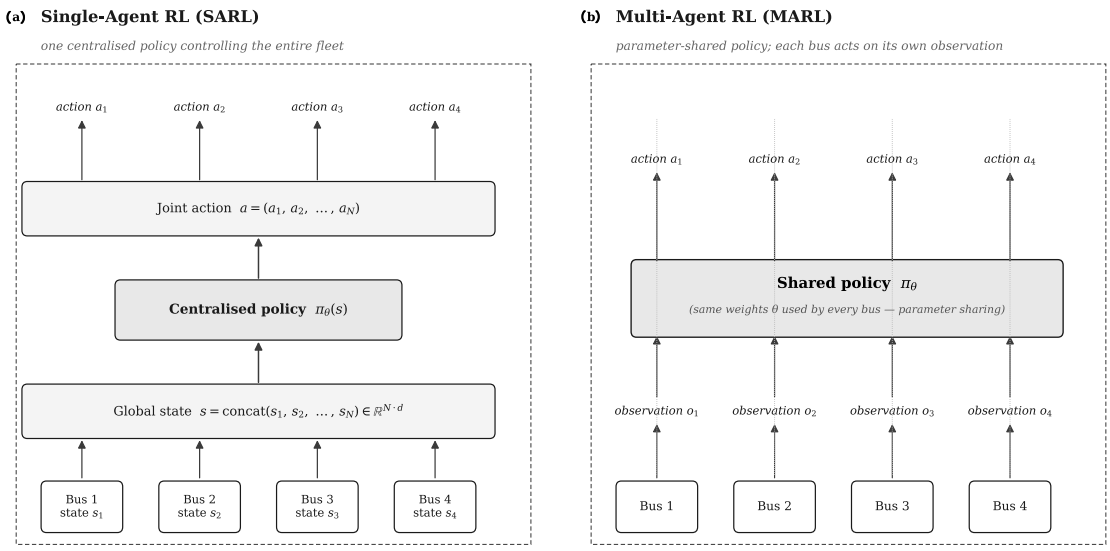

Figure 1.4: Comparison of single-agent and multi-agent formulations. (a) SARL: a centralized policy $\pi_\theta(s)$ consumes the concatenated global state $s \in \mathbb{R}^{N \cdot d}$ and outputs a joint action covering every bus. (b) MARL with parameter sharing: a single set of weights θ is shared across N agents, each acting on its own local observation o_i. Authors' illustration, after the parameter-sharing formulation of Gupta et al. [38]; the combinatorial growth of the centralized single-agent formulation in panel (a) follows Buşoniu et al. [39].

1.2.3 Multi-Agent Reinforcement Learning

In both panels of Figure 1.4, the per-bus state (denoted $s_{i,t}$ in the formal MDP notation of Section 3.2.7, and shown in the figure as the local observation o_i) encodes the bus's current position, forward and backward headways, onboard load, and queue length at its current stop, as defined in full in Section 3.2.7. The action $a_{i,t}$ is the holding-strength and stop-skipping decision the controller emits for that bus, defined in Section 3.2.7. In the SARL panel (a), a single centralized network ingests all N per-bus state vectors concatenated into one global state $s \in \mathbb{R}^{N \cdot d}$ and outputs all N actions simultaneously; in the MARL panel (b), the same shared network weights θ instead process each bus's local state independently, so each agent acts on only its own observation rather than the concatenated global one.

Multi-Agent Reinforcement Learning (MARL) addresses the three limitations above by decomposing decision-making across multiple agents that share the environment. Each limitation has a direct design counterpart in MARL: state-space growth is contained by giving each agent only its own *local* observation rather than the global state; action-space growth is factorized by having each agent select only its own action; and the sample-inefficiency problem is mitigated by sharing one network's parameters across all agents, so that every agent's experience updates the same policy. The problem is typically formalized as a Markov Game built on an underlying Markov Decision Process, with each agent maintaining its own local

observation and acting under a shared or coordinated policy [40–44].

This decomposition introduces a new difficulty: *partial observability*. No individual bus can see the full corridor state; it knows only its own headway, load, and immediate surroundings. A fully decentralized scheme in which each agent both trains and acts on its local observation alone would be unstable, because the other agents' simultaneously evolving policies make the environment non-stationary from any one agent's viewpoint — the same local state can transition to different outcomes depending on what the other agents happen to be learning at that moment.

To address this, many formulations use **Centralized Training with Decentralized Execution (CTDE)**, illustrated in Figure 1.5. In this study, centralization is implemented through one shared learner and replay buffer: each agent contributes a local transition and per-agent reward to the same parameter update. The actor does not require a joint-state input or team-level reward. At deployment, each bus applies the frozen shared weights to its own local observation, so no centralized real-time action selector is required [45–48].

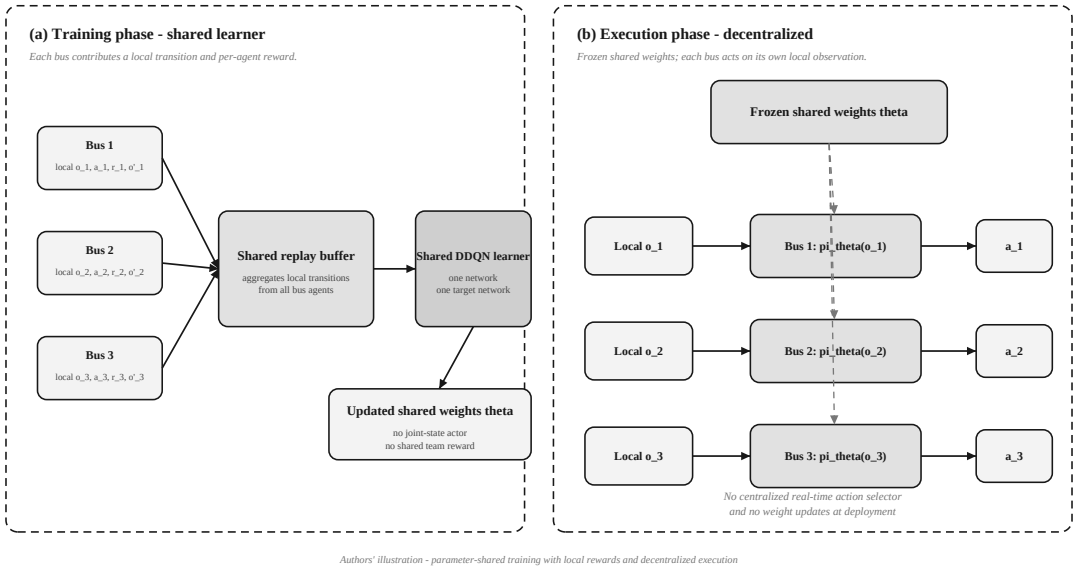

Figure 1.5: (a) During training, agents contribute local transitions and per-agent rewards to one replay buffer that updates a shared set of policy weights θ. (b) At deployment, the weights are frozen and each bus runs an identical copy of π_θ on its own local observation o_i alone, so no centralized real-time controller is required. Authors' illustration, after the centralized-training, decentralized-execution paradigm of Lowe et al. [49].

In the bus-scheduling setting, a parameter-sharing CTDE architecture is fleet-size invariant: the same trained policy applies regardless of how many buses are currently in service, because every bus is an instance of the same agent class acting on its own local headway, occupancy, and queue observations. Adding or removing a bus does not invalidate the trained model. Christianos et al. [34] demonstrate that parameter sharing scales effectively to large agent populations and provides

empirical support for the fleet-size invariance claim.

Recent progress has combined MARL with deep learning algorithms — Deep Q-Networks (DQN), Proximal Policy Optimization (PPO) and its multi-agent variant MAPPO, and Graph Attention Networks (GAT) — to handle high-dimensional observation spaces and complex interactions. These have been applied across vehicle scheduling, signal control, and logistics, generally reporting improvements in routing efficiency, congestion, and throughput in simulation [40–46].

1.2.4 MARL Applied to Bus Scheduling

Within bus scheduling specifically, the MARL literature has accumulated along a recognizable trajectory: first establishing that holding control could be learned at all, then enriching the action and objective space, and most recently beginning to examine policy behavior under disturbance. This trajectory is what the present study extends.

The foundational result is due to Wang and Sun [18], who showed that a cooperative MARL framework could learn an effective bus-holding policy on a single-line corridor and outperform classical headway-equalization rules under idealized stochastic demand. Chen et al. [50] provided an earlier proof-of-concept in the same vein on a corridor with multi-agent reinforcement learning over local headway observations. These early results established the viability of the approach but were limited to the holding action alone, restricting the policy's leverage once bunching had already developed.

Subsequent work enriched the formulation along two axes. Rodriguez et al. [29] extended the action space by combining holding with stop-skipping in a cooperative DDQN framework on a simulated transit corridor with Poisson-distributed passenger arrivals and Gaussian inter-stop travel times, reporting reductions in average passenger waiting time relative to an even-headway holding baseline. Wang and Sun [33] extended the objective space, scaling MARL to multi-line services with a shared corridor and jointly optimizing headway regularity against operational cost under controlled simulated demand and travel-time profiles.

Recent work has increasingly examined policy behavior under disturbance. Wang and Sun's IQNC-M [30] introduced implicit quantile networks and meta-learning for risk-sensitive bus holding, evaluating performance under demand surges and traffic-speed perturbations but not under weather-induced heavy-tailed delays or discrete vehicle breakdowns. In adjacent settings, Katzilieris et al. [51] integrated MADDPG-based traffic signal control with dynamic bus lane access management on a simulated arterial network with deterministic signal cycles; Liu et al. [52] reported DRL-based holding control under stochastic travel time and demand; and Shi et al. [53] reported a distributed DRL bus control system under uncertain interstation travel time and passenger demand, without modeling vehicle breakdowns or weather effects.

Across this trajectory, the simulation environments often assume stationary Poisson passenger arrivals, symmetric inter-stop travel times, and a failure-free fleet. Reported gains are therefore conditional on those assumptions. The CapMetro APC data permit empirical, stop- and time-specific demand, dwell, load, and segment travel-time baselines, while the verified NOAA join supplies observed ordinary-weather exposure. The APC source contains no breakdown field and does not by itself establish an extreme-weather tail, so those two conditions are introduced only as labeled synthetic stressors. A controller calibrated to the public data still requires explicit robustness testing before its gains can be generalized beyond ordinary observed operation.

Table 1.4: Disturbance coverage across MARL bus-scheduling literature and adjacent breakdown-control evidence.

Paper	**Algorithm**	**Environment**	**Disturbances**	**Reported gain**
Wang & Sun (2020) [18]	Cooperative MARL holding	Single-line, idealized	D	Bunching reduction vs. headway rule
Chen et al. (2016) [50]	Early MARL holding	Corridor, idealized	D	Proof-of-concept gains
Rodriguez et al. (2023) [29]	Cooperative DDQN (hybrid action)	Chicago, mesoscopic	T	Wait time vs. headway-control baseline
Wang & Sun (2023) [33]	Multi-objective MARL	Shared corridor, idealized	D	Multi-line bunching reduction
Wang & Sun (2023) [30]	IQNC-M (distributional)	Real-world bus services	S, T	Stable control under extreme events
Katzilieris et al. (2026) [51]	MADDPG	Athens arterial, SUMO	T	Throughput improvement
Liu et al. (2023) [52]	DRL holding	Synthetic single line	D, T	Wait time vs. no-control
Shi et al. (2022) [53]	Distributed DRL	Connected environment	D, T	Integrated control gains
Adjacent breakdown-control evidence (not MARL bus control)				
Cao et al. (2022) [45]	MARL rescheduling	Flatland rail network	B	Adaptation under train malfunctions
Guedes and Borenstein (2018) [54]	Heuristic rescheduling	Real public-transit bus network	B	Recovery after vehicle failures
This study	**DDQN + PS (CTDE)**	**Calibrated CapMetro Route 801 SUMO**	**S, T, W, B**	**To be evaluated**

D = stochastic demand, S = demand surges, T = traffic-speed perturbation, W = weather-induced heavy-tail delay, B = discrete vehicle breakdown.

The two B entries reconcile the breakdown studies as adjacent evidence without misclassifying either one. Cao et al. [45] apply MARL to *train* rescheduling under malfunctions, whereas Guedes and Borenstein [54] apply heuristic real-time rescheduling to public-transit bus failures. The former is MARL but not bus control; the latter is bus failure control but not MARL. Source verification also showed that Shi et al. [53] study uncertain passenger demand and interstation travel time (D, T), not breakdowns. Consequently, none of the B entries in Table 1.4 is a prior MARL bus-scheduling controller.

Table 1.4 summarizes what each study evaluated, what disturbances it modeled, and what it reported. No prior MARL bus paper identified in this review evaluates demand surges, ordinary travel-time variability, heavy-tailed weather stress, and discrete breakdowns together in a public-data-calibrated corridor simulation. Of these classes, observed ordinary rain is available for the CapMetro baseline; severe weather and breakdowns remain synthetic stressors. The algorithmic choice for that setting, DDQN with parameter sharing under CTDE, is justified in Chapter 3 (Section 3.2.7). At each control event, an agent selects a holding strength from $\alpha \in \{0.0, 0.1, 0.2, 0.3, 0.4\}$, scaled by a maximum holding duration, and a binary next-stop skip decision, giving $|A_i| = 10$ actions. The full action-space rationale is given in Chapter 3 (Section 3.2.7).

Demand (D) and empirical traffic-time variability (T) remain active in every evaluation phase. Under the non-ideal regime, the synthetic weather factor (W) is composed multiplicatively with the empirical T draw rather than replacing it (Section 3.2.6). Thus a "W-only" ablation means D+T+W with S and B inactive; it isolates W's incremental effect while retaining the same public-data-calibrated baseline used by every controller.

1.2.5 The Disturbance Gap

The idealized assumptions in Table 1.4 limit how confidently the reported MARL gains can be expected to transfer to real urban corridors. Real corridors violate all three assumptions: demand exhibits non-stationary surges, weather-induced travel times are skewed and heavy-tailed, and individual buses break down stochastically. Wang and Sun themselves identify this as central, noting that existing MARL bus-control studies overlook performance under perturbations and that this becomes critical when transferring models toward real-world deployment [30]. Their IQNC-M framework is among the first to address demand surges and traffic-speed perturbations in bus holding control, but weather-induced heavy-tailed delays and discrete vehicle breakdowns remain outside its tested scenario set.

Two considerations explain why the gap has persisted:

1. **Training instability under heavy-tailed returns.** Prior MARL bus-scheduling

studies have typically used Poisson arrivals and Gaussian travel-time noise because these distributions generally produce more statistically stable training behavior than heavy-tailed disturbances. In reinforcement learning literature, heavy-tailed returns and stochastic gradients are known to introduce convergence difficulties and unstable optimization behavior during policy learning [55, 56].

2. **Lack of temporally aligned anomaly data.** Historical severe-weather records are rarely aligned with operational bus data at the per-segment granularity needed for direct parameter estimation, making it difficult to fit an empirically grounded heavy-tailed disturbance distribution from corridor-specific data alone, especially given the short operational observation windows typically available. *This study addresses the obstacle by adopting the lognormal travel-time parameterization that Patil et al. [37] validated against INRIX freeway data via the Kolmogorov-Smirnov test, and then sweeping the disturbance intensity across a range of validated and extrapolated CV values rather than fitting a single point estimate. This characterizes controller behavior as a function of disturbance magnitude and avoids reliance on a corridor-specific severe-weather sample whose variance cannot be reliably resolved from a short observation window.*

1.2.6 Significance of Closing the Gap

This study contributes the first characterization of MARL bus-scheduling performance across a swept range of disturbance intensities on a calibrated real-corridor simulation, with the disturbance set covering heavy-tailed weather-induced travel-time delays and discrete bus breakdowns that no prior MARL bus-scheduling study has modeled. Understanding policy performance under disturbance matters for two reasons. First, deploying MARL bus-scheduling models on the basis of ideal-simulator benchmarks alone leaves their real-world failure modes unknown until deployment, which is an unacceptable risk profile for a system that carries hundreds of thousands of daily passengers. Second, the relevant performance question for transit operators is not only average-case waiting time under quiet conditions but how a control policy degrades as a disruption intensifies, a characterization that prior MARL bus-scheduling work does not report. Establishing a reproducible stress-testing protocol pairing a calibrated corridor simulation with configurable demand, weather, and breakdown generators therefore contributes both an empirical characterization of performance under disturbance and a reusable evaluation framework for subsequent MARL work in this domain [40, 43, 44, 46, 57].

Chapter 2

Problem Statement

2.1 Rationale

High-frequency bus operation is a sequential control problem: a holding or stop-skipping decision changes the headways, queues, loads, and downstream dwell times encountered by later buses. Fixed timetables cannot adapt after these quantities depart from their planned values [15, 22]. Reinforcement learning is therefore relevant because it learns an action policy from repeated state transitions rather than using a fixed dispatch rule.

The empirical case is CapMetro Rapid Route 801 in Austin, Texas. CapMetro's public APC dataset covers July–December 2021 and contains event timestamps, route and direction codes, stop IDs, boarding and alighting counts, onboard load, dwell time, revenue travel time and distance, event coordinates, and quality indicators [20]. Under identical cleaning rules, the reproducible audit found 455,654 clean stop events for Route 801 and 376,801 for Route 803. Route 801 also had more recorded boardings and more positive-time/positive-distance segments, which justifies its selection as the data-richer primary case. The decision does not imply superior or inferior service quality.

The one-direction study subset uses direction code 6 and contains 229,421 clean stop events, 184 service-day codes, and 29 distinct stop IDs. The direction is not assigned a compass label because a checksum-verified 2021 GTFS snapshot has not yet been acquired. Historical stop names, route shapes, scheduled headways, and capacity values therefore remain gated rather than being borrowed from the current schedule.

NOAA LCDv2 observations make an empirical ordinary-weather exposure feasible [21]. After the documented local-standard-time conversion, the nearest-observation join matched all primary APC events at both Camp Mabry and Austin-Bergstrom

within 90 minutes, including 11,804 Camp Mabry rain-exposed events. Join coverage alone does not establish causality. Ordinary-rain effects must be estimated within comparable segment, time-of-day, and day-type strata; severe weather outside the observed support remains an explicitly synthetic stress test. Vehicle breakdowns are also synthetic because the APC dataset contains no failure-event field.

The resulting problem is to determine whether a parameter-sharing MARL controller, calibrated to observed CapMetro demand, dwell, load, and segment travel-time behavior, remains competitive with fixed-rule controllers when controlled demand surges, weather stress, and breakdowns are introduced without misrepresenting those synthetic variables as measured CapMetro incidents.

2.2 Research Gap

Existing Multi-Agent Reinforcement Learning frameworks for bus scheduling demonstrate measurable improvements in passenger waiting time and headway regularity, but these gains are validated almost exclusively in simulation environments that assume stationary passenger arrivals, symmetric Gaussian travel times, and a failure-free fleet [18, 29, 30]. The performance of MARL bus-scheduling models has not been characterized when these idealizing assumptions are simultaneously violated by (i) stochastic passenger demand surges, (ii) heavy-tailed weather-induced travel-time delays, and (iii) discrete bus breakdowns. Without this characterization, it cannot be determined whether reported MARL gains persist, degrade gracefully, or collapse under realistic operating disturbances, which in turn blocks the transition of MARL bus scheduling from simulation to real urban transit deployment.

The weather-disturbance class (W) in particular was arrived at through the literature survey conducted above: none of the MARL bus-scheduling studies reviewed models heavy-tailed, weather-induced travel-time delays, leaving the W column of the comparison unfilled. Its operational relevance is not assumed but established—rainfall materially reduces average speed and free-flow capacity on comparable corridors (Section 1.1)—and it is made a controllable disturbance through the coefficient-of-variation-driven lognormal travel-time form validated by Patil et al. [37]. Weather therefore enters the study as a defined, parameterized stressor arrived at from the evidence rather than an untested label.

The CapMetro and NOAA sources narrow that gap in two ways. First, they replace assumed baseline demand, dwell, load, and segment travel-time profiles with reproducible public observations. Second, they permit observed ordinary-rain exposure to be separated from synthetic out-of-support stress. They do not supply passenger arrival times, capacity, breakdowns, or an authoritative historical schedule.

The study therefore distinguishes measured calibration inputs, derived variables, simulated passenger states, and synthetic stressors in every claim.

2.3 Research Objectives

2.3.1 Main Objective

To evaluate a parameter-sharing Multi-Agent Reinforcement Learning controller for dynamic bus scheduling in a public-data-calibrated simulation of CapMetro Route 801, direction code 6, under the baseline and non-ideal operating conditions defined in Section 2.5.

2.3.2 Specific Objectives

1. **Environment Construction:** To construct a stochastic traffic simulation environment representing a defined traffic area.

   Expected Output 1.1: A simulation model of a defined operational scope, validated against empirical traffic data to establish an environment that accurately approximates real-world traffic.

   Expected Output 1.2: An environment simulator capable of introducing environmental variability through adjustable parameters to simulate non-ideal operating conditions.

2. **Algorithm Implementation:** To develop and implement a Multi-Agent Reinforcement Learning (MARL) approach for dynamic bus scheduling.

   Expected Output 2.1: A formulated agent architecture defining individual bus units as agents with discrete state and action space with a continuous reward function that punishes irregularities and passenger service delays.

3. **Performance Evaluation Under Ideal and Non-Ideal Conditions:** To evaluate the scheduling performance of the MARL approach under both ideal operating conditions and non-ideal conditions.

   Expected Output 3.1: A quantitative and statistical assessment of MARL scheduling performance under ideal conditions, measuring passenger waiting time, travel time, and headway coefficient of variation, tested against baseline methods.

   Expected Output 3.2: A quantitative and statistical assessment of MARL scheduling performance under non-ideal conditions, measuring passenger waiting time, travel time, and headway coefficient of variation, reported as degradation relative to ideal-conditions performance.

2.4 Significance of the Study

This study contributes both practical and scientific significance, in that order: the operational question comes first, and the methodological contribution follows from how it is answered.

Practical significance. Route 801 recorded 810,309 boardings across 184 service days in the July–December 2021 window (Table 1.1). At that volume a small per-passenger improvement aggregates: a 30-second reduction in mean waiting time, applied across the recorded boardings, corresponds to roughly 6.8×10^3 passenger-hours over the same window. That figure illustrates the scale at which corridor control operates; it is not a performance target, and waiting time in this study is a simulated quantity rather than an observed one. The evaluation is further designed around the question an operator actually faces during a disruption, which is not average-case waiting time under quiet conditions but how far service degrades as conditions worsen—reported here across the baseline, combined-disturbance, and single-disturbance conditions. For CapMetro specifically, the route choice, data exclusions, NOAA join, and unresolved source gaps are documented closely enough that a third party can re-run or contest them.

Scientific significance. The study separates three layers that prior work commonly conflates: empirical ordinary operation, empirically joined weather exposure, and synthetic out-of-support stress. Keeping them distinct is what allows a robustness result to be attributed to the controller rather than to an undeclared modeling choice. The study also contributes the evaluation apparatus itself: a corridor model calibrated to public data, paired with configurable demand-surge, weather-stress, and breakdown generators, evaluated under matched random seeds across controllers. Because the corridor, the disturbance definitions, and the response variables are fixed while the underlying sources remain public, a later algorithm can be assessed on the same basis rather than on a private simulator, so performance claims in this area can be compared instead of reported in isolation.

2.5 Scope and Limitations

Scope. The empirical scope is CapMetro Route 801, direction code 6, during July–December 2021. The simulation uses 29 observed stop IDs and a single simulated operating day per episode. The active fleet size, service horizon, scheduled headway, capacity, and control-stop set are implementation parameters that must

be derived from verified sources or left to be finalized in the implementation phase. Each active bus is an agent that uses the same learned DDQN weights but produces its own action from a local observation.

The experimental activation matrix is fixed as follows. Baseline stochastic demand (D) and ordinary empirical segment travel-time variation (T) are present in training and every evaluation. Training domain-randomizes demand surge (S), weather stress (W), and breakdown (B). Stage A evaluates D+T only, with S/W/B disabled. Stage B evaluates D+T+S+W+B. Three ablations add S only, W only, or B only to the Stage A baseline. Thus W never replaces T, and an ablation labeled "weather only" means D+T+W rather than a deterministic corridor with weather as its sole source of randomness.

Ordinary observed rain is joined from Camp Mabry, with Austin-Bergstrom used for station sensitivity. Temperature, relative humidity, wind, and visibility may enter as control covariates, but only precipitation and rain/weather codes define the primary observed exposure. A lognormal coefficient-of-variation sweep may be used only for severe/extreme stress outside the empirical support and must be labeled synthetic. The breakdown process is likewise synthetic and must be reported separately from observed APC facts.

The MARL policy is compared with No Control, Forward Headway, and Even Headway using simulated passenger waiting time, bus travel time, and headway coefficient of variation over at least 30 matched-seed Monte Carlo runs per evaluation cell. Stage A and Stage B therefore describe evaluation conditions; training itself uses domain randomization across the disturbance classes throughout.

Limitations. (a) APC boardings do not identify passenger arrival times, so passenger waiting time is generated by the simulation and cannot be called directly observed. (b) APC coordinates are stop-event points, not continuous trajectories; calibration therefore uses segment travel time and event-level location quality rather than claiming continuous speed trajectories. (c) The units of `rev_distance` depend on external odometer metadata and remain unresolved; travel-time RMSE is primary until units are verified. (d) Capacity and 2021 schedule semantics require separate authoritative sources. (e) Direction code 6 remains unlabeled until a compatible historical GTFS snapshot is checksum-verified. (f) No breakdown is inferred from missing or delayed APC records.

Delimitations. (g) Feeder-road traffic is not simulated explicitly; the agent observes only route-level headway, load, and queue states, so surrounding-traffic effects enter the model exclusively through the empirical demand and segment travel-time distributions already fit to the corridor. (h) The evaluation is based on repeated matched-seed Monte Carlo runs of a single simulated operating day rather than modeling long-term operational variation across multiple calendar days. (i) MARL

control actions are applied only at the designated control stops selected under the study's stated criteria, and only demand surges, weather stress, and vehicle breakdowns are modeled as disturbance classes; other disturbance types (e.g., traffic accidents, driver behavior variability) are outside the study's scope. (j) The study is scoped to simulation only, with no on-road deployment or sim-to-real transfer claimed.

Chapter 3

Methods and Research Design

This chapter describes the development, training, and evaluation of a Multi-Agent Reinforcement Learning (MARL) model for dynamic bus scheduling under non-ideal conditions. The goal is a reproducible pipeline from the public-data-calibrated CapMetro Route 801 direction-code 6 case—introduced in Section 1.1 and selected as the data-richer corridor in Section 2.1—to measurable controller performance under baseline and disturbed operating conditions.

3.1 Research Design

The study uses a **controlled comparative experimental design** with control strategy (No Control, Forward Headway, Even Headway, MARL) and the activation-matrix condition as factors. Conditions include the D+T Stage A baseline, the D+T+S+W+B combined evaluation, the S/W/B single-disturbance ablations, and the separate synthetic η sweep (here D = baseline demand, S = demand surge, T = ordinary travel-time variation, W = weather stress, and B = breakdown; these five classes are defined in Section 3.2.6, and their on/off status in each condition is given in the activation matrix, Table 3.1). The response variables are simulated mean passenger waiting time, mean bus travel time, and headway coefficient of variation.

Each combination is replicated $N_{\text{runs}} \geq 30$ times using independent Monte Carlo seeds. The unit of analysis is one full simulated operating day on Route 801 direction code 6. Random seeds and disturbance realizations are matched across control strategies within each evaluation cell, so paired performance differences reflect the controller rather than different stochastic draws [55, 56]. Bootstrap intervals are retained for heavy-tailed responses instead of treating the sample-size threshold alone as proof of normality.

Throughout, disturbances play two distinct roles that must not be conflated. During **training**, the full range of stochastic disturbances is always active, the policy is trained under domain randomization [58] so that it learns to act under demand surges, traffic friction, heavy-tailed weather delays, and breakdowns. During **evaluation**, the same generators are used to define two test conditions (ideal and non-ideal) that probe the *already-trained* policy on disturbance realizations it did not see during training but which are drawn from the same generative process. The ideal/non-ideal split is therefore an **evaluation factor, not a training switch**: the agent is never trained on a disturbance-free corridor.

3.2 Methodology

3.2.1 Simulation Environment

The study uses a two-stage simulation architecture. **SUMO** (Simulation of Urban Mobility) [59] serves as the **calibration layer**, providing a microscopic representation of Route 801 from which empirical parameters are extracted. A **custom Python environment built on the PettingZoo Agent Environment Cycle (AEC) API** [60] serves as the **training and Monte Carlo evaluation layer**, where MARL agents interact with a lightweight bus dynamics model parameterized by the calibrated distributions. SUMO is used for corridor calibration rather than for the full learning budget; all training and evaluation runs execute in the lighter event-driven environment.

PettingZoo's AEC API is chosen over three alternatives. First, the single-agent Gymnasium interface [61] cannot natively represent multiple agents acting on independent events. Fitting multi-agent control into a single-agent API requires either treating the entire fleet as one composite agent, which breaks parameter sharing, or coordinating agents manually outside the API, which removes the benefit of using a standard interface. Second, PettingZoo's parallel API assumes all agents step simultaneously [60], which does not match asynchronous bus control where only the bus arriving at a control stop has a decision to make. Third, a fully custom implementation was considered but rejected because it would sacrifice compatibility with established algorithm libraries and complicate reproducibility.

AEC handles the asynchronous case by cycling through one agent at a time rather than stepping all agents in parallel [60]. This matches the approach used by Wang and Sun [30] and Rodriguez et al. [29], both of whom implemented the same asynchronous logic in custom event-driven simulators. Adopting AEC preserves their event-driven dynamics while providing a standardized interface compatible

with parameter-sharing algorithm libraries.

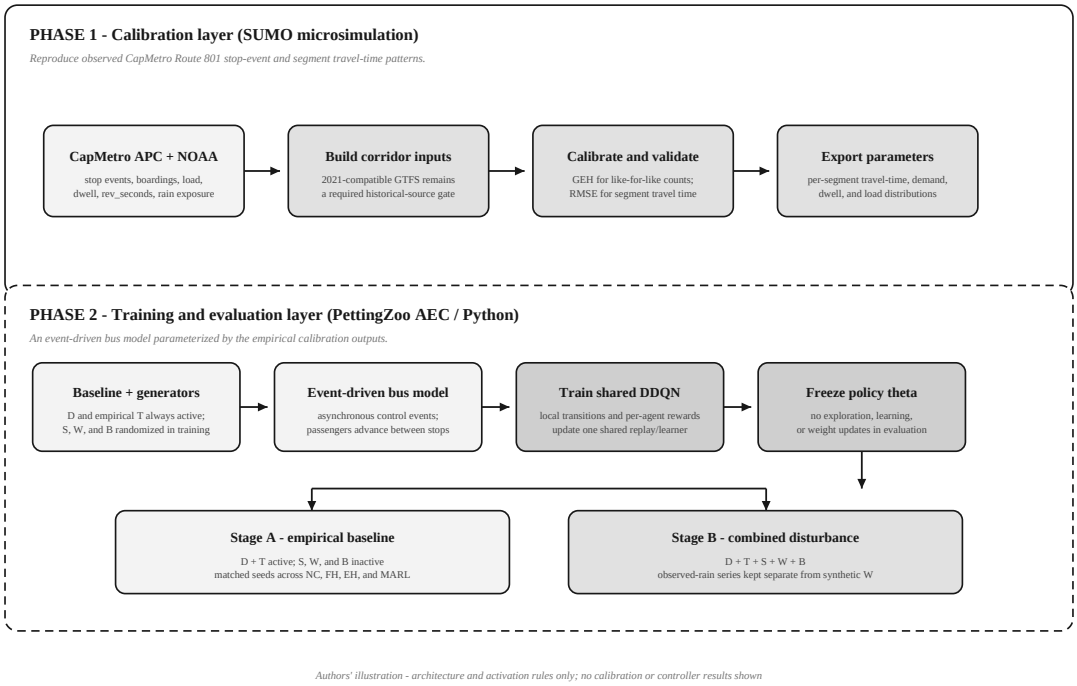

Figure 3.1: The two-phase simulation architecture. Authors' illustration.

SUMO is used for the calibration layer because it integrates with Python through the Traffic Control Interface (TraCI) and supports the count- and travel-time validation protocols described later in this section. The network geometry and schedule-dependent semantics are not borrowed from a current feed: a 2021-compatible GTFS snapshot is required before those inputs are finalized [62].

A pure-Python alternative without SUMO calibration was also considered but rejected: it would lack the empirically grounded per-segment travel-time distributions that the GEH-and-RMSE-calibrated SUMO model provides. The chosen approach uses each tool where it is strongest: SUMO for calibrated microscopic realism during the one-time calibration phase, and Python for fast event-driven Monte Carlo during training and evaluation. This keeps the experiment tractable on accessible hardware, including a single cloud-notebook GPU runtime. Rodriguez et al. [29] and Wang and Sun [30] adopt the same two-layer approach, using event-based custom simulators with calibration parameters supplied from empirical data.

As shown in Figure 3.1, the pipeline proceeds in two phases. First, SUMO is calibrated against empirical bus data, and the resulting per-segment travel time, demand, and dwell time distributions are saved. Second, the Python environment loads these distributions and runs a lightweight event-based bus simulator for MARL training and evaluation. Because the Python environment draws from distributions extracted from the calibrated SUMO model, its statistical behavior matches SUMO

at the level of per-segment travel time, demand, and dwell time. It does not reproduce SUMO's microscopic vehicle-to-vehicle interactions, but those interactions affect the reward function only through their aggregate effect on inter-stop bus travel time, which the calibrated distributions already capture. Rodriguez et al. [29] make the same argument: they trained in an event-based mesoscopic simulator parameterized by empirical corridor data.

3.2.2 Control-Stop Selection

The study operates on Route 801 direction code 6. The reproduced APC subset contains 29 distinct stop IDs. Only a subset is designated as *control stops* where MARL agents may act; the remainder contribute empirical dwell and delay behavior without intervention. Control stops are selected according to four criteria, following Rodriguez et al. [29] and the minimal-intervention principle of Wang and Sun [30]:

1. **Origin terminal.** The route origin is always a control stop, since it is the natural point at which initial headways are set before buses enter the corridor.
2. **Onset of high-demand segments.** The first stop of each high-demand segment is designated a control stop, so that the agent can act before demand-driven irregularity propagates downstream.
3. **Low through-volume preference.** Stops with substantial through-passenger volume (many onboard riders not alighting) are *avoided* as control stops, so that holding actions do not penalize large numbers of passengers already in transit.
4. **No adjacent control hubs.** Where two high-demand hubs are adjacent, only the upstream hub is designated, so that holding penalties do not compound across two consecutive control actions.

These criteria are data-driven: they specify *how* stops are selected from demand and through-volume statistics, without assuming which stops a particular dataset will yield. Once the operational data are processed (Section 3.2.5) and the per-stop demand and through-volume profiles are known, the criteria are applied to produce the control-stop list for the study sub-corridor.

3.2.3 Environment Model Validation

Before MARL training begins, the SUMO layer is verified against held-out CapMetro observations. GEH is applied to hourly stop-event or departure counts where the observed and simulated quantities are defined identically. RMSE is applied primarily to per-segment running time, computed as `rev_seconds` minus the current-stop `dwell_time`: because `rev_seconds` is recorded open-to-open it already includes that stop's dwell, which is modelled separately, so the two are never double-counted. Average speed may be added only after the unit of `rev_distance` is confirmed

from authoritative odometer metadata. Event coordinates support location-quality checks but are not described as continuous trajectories.

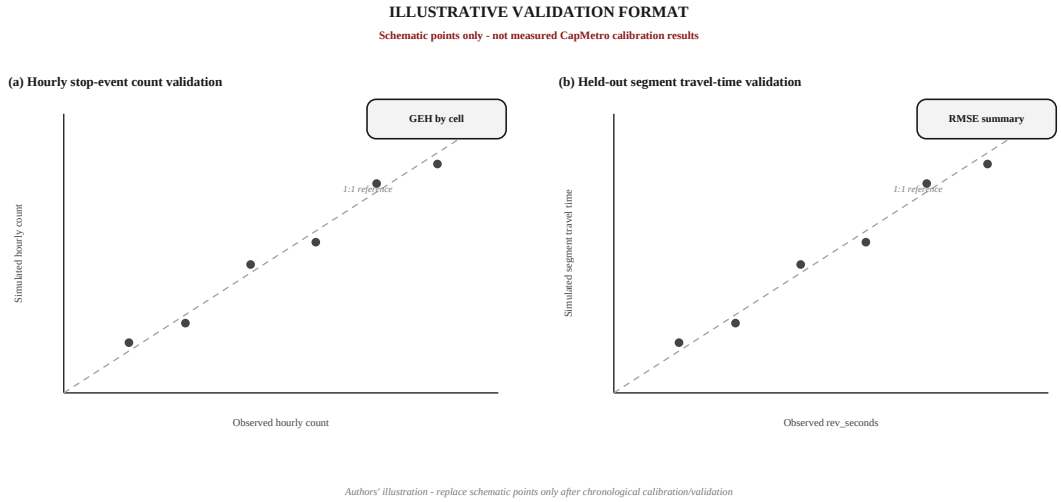

Figure 3.2: Illustrative format of the SUMO calibration output. (a) GEH statistic for identically defined observed and simulated hourly stop-event counts. (b) Simulated segment travel time against held-out empirical `rev_seconds` observations in the RMSE step. Values shown are illustrative only. Authors' illustration.

Bus Volume Validation via GEH Statistic

The GEH statistic, introduced for traffic-modelling acceptance testing in the UK Design Manual for Roads and Bridges [63], measures the discrepancy between simulated and observed hourly bus volumes on individual corridor segments, illustrated in panel (a) of Figure 3.2:

$$GEH = \sqrt{\frac{2(M-C)^2}{M+C}} \tag{3.1}$$

where M is the hourly bus volume from the simulation and C is the observed hourly bus count from empirical operational data. The environment is accepted when $GEH < 5$ for more than 85% of corridor segments, following the FHWA Traffic Analysis Toolbox calibration criterion [64] as applied in recent SUMO calibration work by Patil et al. [37].

Segment Travel-Time Calibration via RMSE

Count calibration alone does not guarantee realistic movement. RMSE evaluates how closely simulated segment travel times match held-out empirical `rev_seconds` observations, illustrated in panel (b) of Figure 3.2:

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n}(x_{obs} - x_{model})^2}{n}} \tag{3.2}$$

where x_{obs} is an observed segment travel time and x_{model} is its simulated coun-

terpart. Parameters such as desired speed and segment friction are tuned on the chronological calibration split, then evaluated once on the held-out split. A speed-based secondary RMSE is permitted only after distance units are verified. Until then, the travel-time measure is the reproducible calibration target.

Real-Time Data Incorporation

In a real-world deployment, forward and backward headways would come from AVL, onboard load from APC, waiting queues from AFC or platform sensing, and weather/incident flags from external services [30]. In this study, APC data calibrate load, boarding, dwell, and segment travel-time distributions; NOAA data supply the observed ordinary-weather exposure; passenger arrival times, queues, and breakdown flags are generated by the simulation and are labeled accordingly.

3.2.4 Operating Conditions

The study uses five disturbance labels: baseline stochastic demand (D), demand surge (S), ordinary empirical travel-time variation (T), weather stress (W), and breakdown (B). D and T remain active in every training and evaluation run. This prevents the baseline from becoming deterministic and prevents W from replacing ordinary empirical travel-time variation.

Training condition. The policy is trained with D+T always active while S, W, and B are domain-randomized over their declared ranges. Weather exposure includes observed ordinary-rain strata where empirical support is adequate and labeled synthetic lognormal stress outside that support.

Stage A baseline evaluation. The frozen policy and all baseline controllers are evaluated under D+T, with S, W, and B disabled. This is an observed-data-calibrated stochastic baseline, not a claim of perfect or "ideal" real-world operation.

Stage B combined-disturbance evaluation. The frozen controllers are evaluated under D+T+S+W+B. Single-disturbance ablations add S only, W only, or B only to D+T. Consequently, "weather-only" denotes D+T+W and "breakdown-only" denotes D+T+B.

Table 3.1 is the authoritative activation matrix. A condition is an environment state; a baseline controller is an algorithmic comparator.

Table 3.1: Authoritative disturbance-activation matrix.

Condition	**D**	**T**	**S**	**W**	**B**
Training	on	on	randomized	randomized	randomized
Stage A baseline	on	on	off	off	off
Stage B combined	on	on	on	on	on
S ablation	on	on	on	off	off
W ablation	on	on	off	on	off
B ablation	on	on	off	off	on

Table 3.2 is the parameter reference for this chapter. Values marked "to be

Table 3.2: Simulation parameter summary: fixed, swept/variable, and derived parameters.

Parameter	**Symbol**	**Value / Range**	**Source**
Fixed simulation parameters			
Simulation horizon	—	Single observed Route 801 service window (to be finalized (data pending): select start/end after historical schedule and timestamp-basis validation)	Section 3.1
Total distinct stop IDs (direction code 6)	M	29	Reproduced CapMetro APC audit
Fleet size (active buses)	N	to be finalized (data pending): derive from concurrent Route 801 vehicle activity and verified schedule	CapMetro APC + historical GTFS
Control stop count	—	to be finalized (design): select from 29 observed stop IDs using Section 3.2.2 criteria after calibration/validation	Section 3.2.2
Scheduled headway	H_0	to be finalized (data pending): 2021-compatible GTFS or schedule record	Historical CapMetro schedule
Bus passenger capacity	—	to be finalized (data pending): verified vehicle/fleet specification	CapMetro fleet specification
Maximum holding duration	ΔT	to be finalized (design) during implementation	Section 3.2.7 (Action Space)
Holding action bins	Ω	$\{0.0, 0.1, 0.2, 0.3, 0.4\}$	Section 3.2.7 (Action Space)
Action space size per agent	$\lvert A_i \rvert$	10 (5 hold $\times$ 2 skip)	Section 3.2.7 (Action Space)
Monte Carlo runs per cell	N_{runs}	≥ 30	Section 3.1
Discount / event-based discount	γ, β	to be finalized (design): tuned during implementation	Section 3.2.7
Swept / variable parameters			
Synthetic severe-weather intensity	η	$\{0.0, 0.3, 0.6, 1.0, 1.3\}$; values above observed support are stress tests	Section 3.2.6
Demand-surge scale	f_d	$\mathcal{N}(1, \sigma_d^2)$ with a design-phase σ_d and support clipped to $[1, 3]$	Section 3.2.6
Traffic-speed stress scale	f_s	$\mathcal{N}(1, \sigma_s^2)$ with a design-phase σ_s and support clipped to $[0.8, 1.2]$	Section 3.2.6
Breakdown rate	λ	to be finalized (design): declared scenario rate (events per bus-hour); APC contains no failure events	Section 3.2.6
Derived empirical/calibration parameters			
Baseline inter-stop travel time	μ	Per-segment, time-of-day, and day-type bin, to be computed	CapMetro `rev_seconds`
Baseline travel-time std. dev.	σ	Per-segment, time-of-day, and day-type bin, to be computed	CapMetro `rev_seconds`
Baseline coefficient of variation	CV_0	σ/μ per segment, to be finalized (data pending)	Computed from dataset
Lognormal shape parameter	σ_{ln}	$\sqrt{\ln(\eta^2 + 1)}$ (Eq. 3.5)	Method of moments
Lognormal unit-mean factor location	μ_{ln}	$-\sigma_{ln}^2/2$ (Eq. 3.6)	Method of moments

finalized" are intentionally deferred at this proposal stage and will be set during the implementation phase: those noted *data pending* await a verified external source or an unfinished empirical derivation, while those noted *design* are experimental choices, several fixed through the Expected Output 2.1 sensitivity analysis. Marking them explicitly keeps unsupported target values from being mistaken for observed CapMetro quantities. The 29-stop count is reproduced from the clean direction-code 6 subset. The demand and speed clip supports are not standard deviations: σ_d and σ_s remain separate distribution parameters and cannot be replaced by their clip bounds.

3.2.5 Data Processing

The MARL framework is trained and evaluated on a single stream of input data pre-processed before training: corridor operational data used to calibrate SUMO and to derive baseline parameters for the stochastic generators. A second stream, the synthetic state observations generated by the Python environment at runtime, is produced on-the-fly during training and consumed once per control event.

Required Datasets

1. **CapMetro APC event data.** The official July–December 2021 Socrata asset `im6q-3pc9` contains 47 text-typed columns [20]. The primary query keeps Route 801 records for which `current_route_id=route_id`, `import_error=0`, `import_trip_error=0`, `bs_id`$\neq 0$, and `direction_code_id=6`. The checksum-recorded result contains 229,421 rows, 184 service-day codes, and 29 distinct stop IDs. It supplies event time, trip/vehicle keys, boardings, alightings, load, dwell, segment travel time/distance, coordinates, and quality flags.

2. **NOAA LCDv2 weather.** Camp Mabry (USW00013958) is the primary station and Austin-Bergstrom (USW00013904) is the sensitivity station [21]. NOAA timestamps are local standard time with no daylight-saving adjustment. They are interpreted as UTC−06:00, converted to `America/Chicago`, and joined to APC event time by nearest observation within 90 minutes. Both stations matched all 229,421 primary events in the feasibility audit; 11,804 events matched a Camp Mabry rain flag.

3. **Historical schedule and fleet metadata.** A 2021-compatible GTFS snapshot is required for authoritative direction labels, stop names, route shape, and scheduled headway; a verified fleet source is required for capacity. The current GTFS asset [62] is not substituted for a historical snapshot. Until those gates close, direction 6 remains code-only and schedule/capacity parameters remain to be finalized (data pending).

Table 3.3 summarizes how each verified CapMetro and NOAA field maps to the

quantity derived from it and the role, and limitation, that quantity carries in the model.

Table 3.3: Verified CapMetro/NOAA fields and their modeling roles.

Source fields	Derived quantity	Model role and limitation
`apc_date_time`, `transit_date_time`	Event instant and service day	Chronological split and weather join; APC timezone remains a declared Austin-wall-clock assumption
`ext_trip_id`, `vehicle_id`, `act_trip_start_time`, `actual_sequence`	Trip-day identity and event order	Headway/segment construction; no single field is assumed globally unique
`bs_id`, direction code	Stop/segment sequence	29 stop IDs for code 6; names and compass label require historical metadata
`ons`, `offs`, `max_load`	Boarding, alighting, and load profiles	Demand/load calibration; not passenger arrival time or capacity
`dwell_time`	Door-open to door-close seconds	Dwell distribution; controller holding is simulated separately
`rev_seconds`, `rev_distance`	Per-segment running time (`rev_seconds` is open-to-open, so the current-stop dwell is netted out) and possible average speed	Travel-time RMSE is primary; distance unit remains unresolved
`veh_lat`, `veh_long`, `quality_indicator`	Stop-event coordinates and GPS quality	Event-level QA; not a continuous trajectory
NOAA precipitation and present-weather code	Observed ordinary-rain flag	Primary weather exposure after time alignment
NOAA visibility, temperature, humidity, wind	Weather covariates	Controls/sensitivity variables, not primary exposure by default

Passenger arrival timestamps, waiting time, capacity, and breakdown events are absent from the APC file. They are simulated or separately sourced and are never described as directly observed. The pooled wet/dry travel-time medians produced by the feasibility audit are descriptive only; causal or calibration multipliers require segment, time-of-day, and day-type adjustment.

Data Pre-Processing Pipeline

Pre-processing proceeds in three stages.

Stage 1: acquisition, integrity, and time normalization. The pipeline records the Socrata query, row count, 47-field schema, file size, and SHA-256 hash. It applies the declared route/error/stop/direction filters before modeling. APC compact timestamps are parsed as Austin wall-clock time; NOAA LCDv2 local-standard timestamps are assigned fixed UTC−06:00 and converted to `America/Chicago`. Ambiguous fall-back APC times are counted and assigned a declared fold rather than silently duplicated. Records with invalid timestamps or non-positive segment time/distance are excluded only from the derivation that requires those values, not from unrelated demand/dwell summaries.

Stage 2: sequence construction and empirical extraction. Events are ordered using

service day, trip/vehicle keys, actual trip start, and actual sequence. Per-(stop, time-of-day, day-type) boarding, alighting, load, and dwell distributions are estimated. Per-(segment, time-of-day, day-type) running-time distributions—`rev_seconds` net of the current-stop `dwell_time`, since `rev_seconds` is measured open-to-open—yield μ, σ, and CV_0. Because the source spans 184 service-day codes, weekday/weekend and time-bin support is assessed empirically rather than assuming a two-week window.

Stage 3: weather alignment and controlled estimation. Each APC event is assigned the nearest Camp Mabry observation within 90 minutes and a Bergstrom sensitivity match. Primary exposure uses precipitation and present-weather rain codes; visibility is secondary, and temperature, humidity, and wind are candidate controls. Any ordinary-rain multiplier is estimated within segment, time-of-day, and day-type strata. The unadjusted pooled wet/dry difference is not used as a causal multiplier.

Stage 4: chronological calibration/validation split. Earlier service days tune SUMO and the empirical parameter files; later held-out days assess stop-event counts and segment travel-time RMSE. No held-out record is used to choose calibration parameters.

The MARL agents are not trained on these raw datasets directly. They are trained on synthetic state vectors $s_{i,t}$ produced by the Python environment at each control event. Each $s_{i,t}$ is assembled at runtime by querying the analytical bus model and is consumed once by the policy network before being discarded. There is no persistent training dataset of state-action transitions on disk other than the standard replay buffer used by the learning algorithm.

3.2.6 Stochastic Disturbance Generators

Disturbance Classes and Independence This study distinguishes five disturbance classes, denoted D, S, T, W, and B:

- **Stochastic demand (D):** the baseline, always-present day-to-day randomness in passenger arrivals, drawn from the calibrated per-(stop, time-of-day) demand distributions (Section 3.2.5). D is not a disturbance layered on top of a deterministic baseline; it *is* the baseline stochastic environment, present in every run regardless of which other generators are active.

- **Demand surge (S):** an episode-level multiplicative scaling factor, with standard deviation σ_d (Table 3.4), that amplifies baseline boarding rates above their empirical mean. S is the controlled experimental variable; D is always present, and S is what is added on top of it. Setting $\sigma_d = 0$ removes the surge and leaves only baseline demand variability (D).

- **Ordinary travel-time variation (T):** the always-present empirical per-(segment, time-of-day, day-type) running-time distribution derived from `rev_seconds` (net of the current-stop dwell). An optional episode-level stress scale f_s may broaden this baseline, with its standard deviation σ_s selected separately from the $[0.8, 1.2]$ clip support.

- **Weather-induced delay (W):** an additive experimental layer. Within observed support, W is estimated by comparing rain-exposed and dry observations within segment/time/day strata. Outside that support, W is a labeled synthetic lognormal stress with coefficient of variation η. W composes with rather than replaces T.

- **Discrete bus breakdown (B):** a Poisson-distributed discrete event, with rate λ, that permanently removes one bus from the active agent set for the remainder of the simulated day.

The generators that produce S, W, and B are sampled independently conditional on the always-active D and T baselines: no causal chain links them within the simulation. A breakdown does not trigger a demand surge or weather delay, and weather does not induce a mechanical failure. This factorial choice supports attribution in the single-disturbance ablations; it is not a claim that real disturbances are causally independent.

Four stochastic generators inject variability into the Python environment. Generators (i) and (ii) follow the perturbation framework of Wang and Sun [30]; the weather generator's heavy-tailed lognormal formulation follows Patil et al. [37]; the breakdown generator follows the rescheduling formulation of Cao et al. [45]. Table 3.4 collects the symbols used across this section and the MARL formulation that follows.

Random Seed Control

A stored master seed initializes named NumPy child streams for D, S, T, W, B, passenger arrivals, and policy exploration. Separating streams ensures that enabling W does not shift the random draws assigned to demand or breakdowns. Within an evaluation cell, the same master-seed list and generator substreams are reused across all four controllers, so run k is a matched counterfactual comparison [55]. Seeds differ across replications. The minimum $N_{\text{runs}} \geq 30$ does not replace distribution diagnostics; bootstrap intervals are used for heavy-tailed responses [56].

Passenger Demand

The baseline empirical transit demand is perturbed each episode by a scaling factor sampled from $\mathcal{N}(1, \sigma_d^2)$ and clipped to $[1, 3]$, following the Gaussian demand-scaling mechanism of Wang and Sun [30]; the clip bounds $[1, 3]$ are this study's

Table 3.4: Notation used in the stochastic generators and the MARL formulation.

Symbol	**Definition**	**Domain / units**
η	Synthetic out-of-support weather-stress intensity; sets the coefficient of variation of the added lognormal travel-time stress. $\eta = 0$ disables this synthetic layer.	dimensionless
σ_d	Standard deviation of the per-episode passenger-demand scaling factor. $\sigma_d = 0$ fixes demand at its baseline mean.	dimensionless
σ_s	Standard deviation of the per-episode traffic-speed scaling factor. $\sigma_s = 0$ fixes cruising speed at its baseline mean.	dimensionless
λ	Rate parameter of the Poisson process governing bus breakdowns. $\lambda = 0$ disables breakdowns.	events/hour
μ	Baseline (empirical) mean inter-stop travel time for a given segment and time-of-day bin.	seconds
σ	Baseline standard deviation of inter-stop travel time.	seconds
CV_0	Baseline coefficient of variation, $CV_0 = \sigma/\mu$, used to anchor the disturbance generators.	dimensionless
μ_{ln}, σ_{ln}	Location and shape parameters of the lognormal weather-delay distribution (method-of-moments).	—
$h^-, \hat{h}^+$	Forward headway (gap to bus ahead) and estimated backward headway (gap to bus behind).	seconds
H_0	Desired (scheduled) headway.	seconds
$\alpha_{i,t}$	Holding-strength action of agent i, $\alpha_{i,t} \in \Omega = \{0.0, 0.1, 0.2, 0.3, 0.4\}$, scaled by ΔT.	dimensionless
ΔT	Maximum holding duration that $\alpha_{i,t}$ scales.	seconds
γ, β	Discount factor and event-based discount rate ($e^{-\beta t}$ replaces γ for random inter-event times).	dimensionless
θ	Weights (parameters) of the online neural network (shared policy).	dimensionless
θ^-	Weights (parameters) of the target neural network in DDQN.	dimensionless
N	Number of active bus agents.	count
N_{runs}	Number of independent Monte Carlo replications per controller/condition cell.	count

own design choice rather than a value taken from that work. The asymmetric clip focuses the test on demand surges rather than symmetric variation, since demand drops produce lightly loaded conditions that do not stress-test the controller. The upper bound of 3, corresponding to roughly a tripling of baseline boarding rates, is this study's own design choice (to be revisited during implementation) rather than a value drawn from prior work. Sampling occurs at the start of each simulation run, producing varied demand profiles across episodes. In implementation, the scaling factor $f_d \sim \mathcal{N}(1, \sigma_d^2)$ is sampled once per episode at initialization and applied uniformly to every per-stop, per-time-of-day arrival rate for the duration of that simulated operating day, so all stops experience the same proportional demand shift within a single run while the shift itself varies across runs.

Traffic Delays

Ordinary inter-stop travel time is sampled from the empirical T distribution for the current segment, time-of-day, and day-type stratum. If a separate corridor-wide stress test is retained, $f_s \sim \mathcal{N}(1, \sigma_s^2)$ is sampled once per episode and clipped to $[0.8, 1.2]$ [30]; σ_s is a design-phase parameter and is not equal to either clip bound. T remains active when W or B is enabled. Thus a weather run preserves ordinary empirical variability and applies its weather layer on top, avoiding the previous replacement/double-definition conflict.

Weather-Induced Anomalies

The weather design has two explicitly separated layers. **Observed ordinary weather** uses the timestamp-aligned NOAA records. Rain exposure is defined primarily from precipitation and present-weather rain codes at Camp Mabry, with Austin-Bergstrom as a sensitivity station. For each segment/time-of-day/day-type stratum with adequate wet and dry support, the implementation estimates a regularized multiplier $m_{s,b,d}(w)$ relative to its dry baseline. Temperature, humidity, wind, and visibility may enter the estimation as controls. No pooled wet/dry median is used as a causal multiplier.

Synthetic severe/extreme weather is used only outside the support of the observed strata. It applies a unit-mean lognormal factor F_W to a draw T_0 from the always-active empirical T distribution. This preserves the ordinary segment variation instead of replacing it:

$$T_W = T_0 \, m_{s,b,d}(w) \, F_W. \tag{3.3}$$

For an exclusively synthetic stress cell, $m_{s,b,d}(w) = 1$. The controlled parameter η is the coefficient of variation of F_W, not a rainfall intensity or a named meteorological category. The lognormal form represents right-skewed travel-time stress and follows the distributional rationale of Patil et al. [37]:

$$CV = \eta \tag{3.4}$$

The lognormal shape and scale parameters follow the method-of-moments derivation:

$$\sigma_{ln} = \sqrt{\ln(\eta^2 + 1)} \tag{3.5}$$

$$\mu_{ln} = -\frac{\sigma_{ln}^2}{2} \tag{3.6}$$

The factor is drawn as $F_W \sim \text{LogNormal}(\mu_{ln}, \sigma_{ln})$, for which Equations (3.5)–(3.6) give $\mathbb{E}[F_W] = 1$ and $CV(F_W) = \eta$. Patil et al. [37] validate a CV-driven lognormal travel-time form against SUMO/INRIX-based urban arterial scenarios. This study uses that form for a declared extrapolative stress factor; it does not claim that Patil et al.'s CV values correspond to Austin rainfall categories.

The synthetic factor is swept over $\eta \in \{0.0, 0.3, 0.6, 1.0, 1.3\}$. Values through 1.0 span the range evaluated by Patil et al.; 1.3 is a deliberate extrapolation. None is assigned a rain-rate label. Observed ordinary-rain experiments are reported by their NOAA exposure definition rather than by η. Table 3.5 summarizes the basis for each swept intensity value.

Table 3.5: Basis for each synthetic weather-stress intensity value.

η	**Basis**
0.0	Synthetic factor off; empirical T remains active
0.3	Within the published CV range; synthetic variability stress
0.6	Within the published CV span; synthetic variability stress
1.0	Upper end of the published CV range; synthetic severe stress
1.3	Beyond the published range; deliberate extreme extrapolation

The mapping from η to named weather severity is not established. The sweep is reported as synthetic travel-time variability, whereas ordinary rain is evaluated through the empirical NOAA join. Keeping the two layers separate prevents observed rain, literature-shaped stress, and out-of-range extrapolation from being presented as one calibrated variable.

Bus Breakdowns

Unlike the perturbations above, a bus breakdown is a *discrete event* that removes a vehicle from active simulation. Cao et al. [45] model Poisson train malfunctions in an adjacent MARL rescheduling setting, while Guedes and Borenstein [54] model serious public-transit vehicle failures in heuristic real-time rescheduling. These sources support the event/removal semantics, not an empirical CapMetro failure rate; the APC data contain no breakdown field. Breakdowns are therefore a labeled synthetic

stressor triggered at random times from a Poisson process [65] with a configurable scenario rate λ (Table 3.4). In implementation, at each discrete simulation timestep of length dt, a Bernoulli trial with probability $\lambda \cdot dt$ is evaluated independently for each active bus; a success removes that bus from the active agent set for the remainder of the simulated day. When a breakdown occurs at bus b_k, b_k is removed from the active agent set, leaving an enlarged headway between the trailing bus b_{k-1} and the leading bus b_{k+1}. The passengers that b_k would have served accumulate at the stops ahead of b_{k-1}, so the trailing bus inherits this residual demand as it advances.

A breakdown does not override any surviving agent's policy; it changes the state and reward landscape. Removing b_k enlarges the gap between the trailing and leading buses, and passengers that would have boarded b_k continue accumulating in the simulated queues. A trailing agent may choose to hold less or skip a stop to close the gap, but a skipped stop serves neither boarding nor alighting passengers in that event and therefore incurs the waiting/skip penalty. The environment never injects a recovery action: every hold/skip decision originates from π_θ.

The Poisson assumption is a tractable approximation for the single-day simulation horizon, over which vehicle age, maintenance schedule, and mileage since last service can be treated as constant.

3.2.7 MARL Architecture and MDP Formulation

This subsection builds the formal learning framework in two steps. It first introduces the Bellman recursion that underlies value-based reinforcement learning, then specifies the Markov game tuple, the per-agent observation and action spaces, the reward-function design, and the proposed learning algorithm. The justification for choosing a multi-agent formulation over a single-agent alternative is presented in Chapter 1.

Bellman Foundation

At the core of reinforcement learning is the dynamic programming principle established by Bellman [66]. The agent's objective is to maximize the expected sum of discounted future rewards. The state-action value function $Q^\pi(s, a)$, denoting the expected return from taking action a in state s and following policy π thereafter, satisfies the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E}\left[r(s, a) + \gamma \max_{a'} Q^*(s', a') \mid s, a\right] \tag{3.7}$$

where s' is the state reached after applying action a, $r(s, a)$ is the immediate reward, and $\gamma \in [0, 1)$ is the discount factor. This equation is relevant here for two reasons. First, it shows that a long-horizon control policy can be learned by recursively backing up local one-step rewards, making it computationally feasible to evaluate the downstream effects of a holding or skipping decision without enumerat-

ing every future stop on the route. Second, in the asynchronous bus-control setting, the time between two consecutive control events is itself random. Following Bradtke and Duff [67], the discount factor in Eq. (3.7) is replaced by an event-based discount $e^{-\beta t}$ that scales with the elapsed time between events. Each agent's learning target is therefore an event-adjusted version of Eq. (3.7), not a fixed-tick MDP target. By convention throughout this chapter, γ denotes the nominal discount factor as it appears in the canonical Bellman and DDQN equations, while $e^{-\beta t}$ is the event-based discount actually applied in the asynchronous setting; wherever γ appears in a value-function equation, it is instantiated by $e^{-\beta t}$ at runtime.

Markov Game Formulation

Following Littman [68], the bus fleet control problem is formulated as a multi-agent extension of a Markov Decision Process, a Markov game defined by the tuple $G = \langle N, S, A, P, R, \gamma \rangle$:

- N: number of bus agents.
- S: joint state space.
- A: joint action space.
- P: state-transition function $P(s_{t+1} \mid s_t, a_t)$.
- R: reward function.
- $\gamma \in [0, 1)$: discount factor.

Because bus control is asynchronous and event-driven, agents act only upon arrival at a control stop. The formulation maintains a per-agent local transition $P_i : S_i \times A_i \to S_i$, with S_i the local observation of agent i [29, 30].

Agents (N)

The decision-making agents are the individual bus units $i \in N$ operating along the corridor. To avoid combinatorial blow-up and to keep the system invariant to fleet size, all agents share a single neural-network policy under a **parameter-sharing** architecture and act on their own local observations at deployment [29, 30, 34]. Parameter sharing means that one set of network weights serves every bus; each bus computes its own action from its own observation, but learning updates from any bus improve the policy for all.

State Space and Local Observations (S_i)

The local observation $s_{i,t}$ at agent i on arrival at a control stop is a vector composed of:

- **Spatial location:** index of the current control stop.

- **System regularity:** forward headway h^- and estimated backward headway $\hat{h}^+$, both available from AVL feeds in deployment [30].
- **Passenger demand:** onboard passenger count and estimated number of passengers waiting at the stop, available from APC/AFC systems [30].
- **Environmental flags:** encoded indicators for the current disturbance intensity and any active downstream incident or breakdown.

Table 3.6: Agent observation vector: features, symbols, and data sources.

Feature	Symbol	Deployment Source	Simulation Source
Control stop index	—	Route map (static)	Hardcoded stop list
Forward headway	h^-	AVL feed	Event-driven bus model
Estimated backward headway	$\hat{h}^+$	AVL feed	Event-driven bus model
Onboard passenger count	—	APC system	Running tally in bus model
Waiting passenger count at stop	—	AFC terminal / platform sensors	Simulated stop queue; not an APC field
Weather exposure/stress flag	w (encoded)	Weather API / public incident alert	Joined NOAA exposure or labeled synthetic W parameter
Downstream incident / breakdown flag	b (binary)	Incident management system	Breakdown generator output

As Table 3.6 shows, the environment generates each runtime observation from its evolving simulated state. Empirical APC/NOAA records calibrate the distributions and exposure strata before training; raw sensor rows are not streamed into the policy during an episode.

Action Space (A_i)

The agent's action $a_{i,t} = \pi_\theta(s_{i,t})$ combines two discrete components, following Rodriguez et al. [29]:

- **Holding control:** a holding-strength parameter $\alpha_{i,t} \in \Omega = \{0.0, 0.1, 0.2, 0.3, 0.4\}$, multiplied by a maximum holding duration ΔT to yield seconds held; $\alpha_{i,t} = 0$ means no holding.
- **Stop-skipping:** a binary decision to bypass the next stop, allowing headway recovery at an explicit passenger-service cost. The policy chooses the action from local delay, load, and queue state; no capacity threshold is assumed until vehicle capacity is verified. Skipping is disallowed at the origin terminal and is not permitted if the immediately preceding trip already skipped that stop, so no stop is skipped by two consecutive trips.

The full action set is the Cartesian product of these two components: $|A_i| = 5 \times 2 = 10$ discrete actions per control event, allowing the agent to select a holding

strength and a skip decision independently at each control event. This is a broader action space than Rodriguez et al. [29], whose combined holding-and-skipping controller (DDQN-HA) instead selects among six *mutually exclusive* actions: five holding strengths $\Omega = \{0.0, 0.1, 0.2, 0.3, 0.4\}$ (with $\omega = 0$ already covering the no-holding case) plus a single skip action. The discretized holding-strength set Ω adopted here matches theirs exactly. A continuous holding parameter $\alpha \in [0, 1]$ was considered, following Wang and Sun [30], but rejected for two reasons. First, continuous actions require actor-critic algorithms, whose training instability compounds across the swept-disturbance evaluation budget. Second, real driver compliance with holding instructions is itself imperfect: Rodriguez et al. [29] model non-compliant drivers as departing after only 60–80% of the instructed holding time, so continuous precision in α is not meaningful at deployment.

Reward Function (R) and Objective

The infinite-horizon objective of agent i is the expected cumulative event-discounted return

$$R_i = \mathbb{E}\left[\sum_{k=0}^{\infty} e^{-\beta t_k}\, r_{i,t+k}\right], \tag{3.8}$$

where t_k is the elapsed time to the k-th future control event, consistent with the event-based discount introduced earlier.

The **structure** of the per-event reward $r_{i,t+k}$ — its component terms and the operational priorities they encode — is defined in this chapter. Its **exact weighting and sensitivity analysis** are a research objective of this study (Specific Objective 2, Expected Output 2.1), to be finalized during implementation rather than inherited from prior work. This separation is deliberate: the reward *form* is grounded here, while the tuning of its coefficients is treated as an empirical question. The reward formulation must balance three competing operational priorities: (i) headway regularity, since bunching is the primary failure mode of bus corridors; (ii) passenger waiting time at served stops, which is the most direct passenger-experience metric; and (iii) a penalty term that discourages degenerate skipping behavior, since an unconstrained skip action could otherwise reduce headway variance by refusing to serve passengers. Existing MARL bus-control reward formulations span a range of trade-offs among these priorities, from headway-coefficient-of-variation forms for holding-only action spaces [30, 33] to passenger-time forms for combined holding-and-skipping action spaces [29]. This study establishes the overall reward structure for the hybrid action space by defining the three reward components and their additive formulation, while treating the corresponding weighting coefficients, together with their sensitivity analysis, as the implementation-phase deliverable under Expected Output 2.1. Although the component structure is fixed at this stage, the

coefficients remain as placeholders to be determined during implementation through experimental evaluation.

The reward is computed independently for each agent at every control event rather than as a shared team-level objective. Accordingly, $r_{i,t}$ represents the reward assigned to agent i and is stored as that agent's transition in the shared replay buffer (Training and Execution Protocol, step 4). Each bus is therefore evaluated based on the consequences of its own action, even though all agents learn from a common shared network. Since the reward is derived from locally observable quantities already contained in the agent's observation, particularly the forward and backward headway measurements, the resulting signal remains aligned with corridor-wide service regularity without requiring a centralized reward computation during execution.

The overall reward function is expressed as the weighted sum of three penalty terms:

$$r_{i,t+k} = -w_1 \cdot (\text{headway-irregularity term}) - w_2 \cdot (\text{waiting-time term}) - w_3 \cdot (\text{skip-degeneracy term}), \tag{3.9}$$

where w_1, w_2, and w_3 denote the weighting coefficients to be determined through the Expected Output 2.1 sensitivity analysis. Each component is formulated as a non-positive penalty, allowing the agent to maximize its cumulative return in Eq. (3.7) by minimizing headway irregularity, passenger waiting time, and unnecessary stop-skipping behavior. Consequently, this chapter establishes the reward formulation and its optimization objective, while the specific mathematical expressions and coefficient values are reserved for the implementation and evaluation phase.

Proposed Learning Algorithm

The fully discrete action space ($|A_i| = 10$) makes value-based learning a natural fit. This study adopts **Double Deep Q-Network (DDQN) [69] with parameter sharing under centralized training and decentralized execution (CTDE)**. Centralization here means that one learner samples per-agent local transitions from a shared replay buffer and updates one shared online/target network. It does not mean that the DDQN actor consumes a joint state or shared team reward: each transition contains agent i's local observation and per-agent reward during both training and execution. The choice follows Rodriguez et al. [29], who used a DDQN-based framework for cooperative holding-and-skipping control on a comparable corridor.

DDQN is chosen over three categories of alternatives. *Plain DQN* [70] overestimates Q-values in noisy environments [55, 69], which the high-variance disturbance regime in this study would amplify. DDQN's double estimator addresses this by de-

coupling action selection from evaluation [69]. *Actor-critic methods such as MAPPO and MADDPG* are compatible with continuous action spaces but unnecessary given the discrete action-space decision above; they would also introduce training instability that compounds across the swept-disturbance evaluation budget. *Distributional methods such as Wang and Sun's IQNC-M* [30] model the full return distribution rather than its expectation and offer strong performance on this problem class, but their meta-learning training procedure adds implementation risk beyond the scope of a thesis-scale project; IQNC-M is noted as a comparison target for future work.

Two enhancements from the recent bus-control literature will be tested during implementation: extending agent awareness to include neighboring-trip information in the observation and reward, as in Rodriguez et al.'s DDQN-HA variant [29], and credit assignment via inductive graph learning, as in Wang and Sun's CAAC framework [30]. The final architecture will be selected based on training stability and validation-set passenger waiting time after a fixed training budget.

Double Deep Q-Network Foundation

The Bellman optimality equation defines the value $Q^*(s, a)$ that an ideal controller would assign to each state-action pair, but it cannot be solved exactly for a state space as large as the bus-control problem, where the observation is a continuous vector of headways, loads, and queues. Two approximations bridge this gap. Q-learning [66, 68] replaces the exact solution with an iterative update that nudges a stored estimate $Q(s, a)$ toward the one-step Bellman target $r + \gamma \max_{a'} Q(s', a')$ after each transition. The Deep Q-Network (DQN) [70] replaces the lookup table with a neural network $Q(s, a; \theta)$, so that the value function generalizes across the continuous state vectors the bus environment produces. DQN trains θ by minimizing the temporal-difference loss between the network's prediction and a target computed from a periodically frozen copy of the network, with transitions drawn from an experience replay buffer to decorrelate consecutive samples.

DQN has a known issue: it systematically overestimates action values [69]. The cause is the single max operator in the target. Because the same network both selects the next action ($\arg\max_{a'}$) and evaluates it ($\max_{a'} Q$), noise in value estimates is amplified the max preferentially picks whichever action happens to be overvalued, and that inflated value propagates into the target. In a low-variance environment this bias is manageable, but the swept-disturbance regime in this study generates heavy-tailed returns that feed the overestimation mechanism directly.

The Double Deep Q-Network (DDQN) [69] removes this bias by decoupling action selection from action evaluation. The online network (weights θ) selects the next action, but a separate target network (weights θ^-) evaluates that action's value:

$$y^{\mathrm{DDQN}} = r(s, a) + \gamma\, Q\Big(s',\, \arg\max_{a'} Q(s', a'; \theta);\; \theta^-\Big)\,, \tag{3.10}$$

For the overestimation to persist, both networks would have to overvalue the same action simultaneously, which is less likely than a single network doing so alone. The network is trained by minimizing the squared temporal-difference error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y^{\mathrm{DDQN}} - Q(s,a;\theta)\right)^2\right], \tag{3.11}$$

where $\mathcal{D}$ is the experience replay buffer and the target weights θ^- are held fixed for a number of updates (or slowly tracked toward θ) so that the regression target does not shift at every step.

Three properties make DDQN appropriate for this study. First, the discrete action space ($|A_i| = 10$) is the setting value-based methods handle well: the arg max in Eq. (3.10) is a finite enumeration over ten options. Second, the double estimator addresses the overestimation that high-variance disturbance returns would otherwise amplify. Third, DDQN inherits DQN's experience-replay and target-network machinery, which provides sufficient training stability without the auxiliary mechanisms that actor-critic methods require; this point is discussed further in Section 3.3. The event-based discount $e^{-\beta t}$ replaces the fixed γ in Eq. (3.10) when the target is computed in the asynchronous setting, so the DDQN target inherits the same event-adjustment as the underlying Bellman recursion. The combination of DDQN with parameter sharing and the CTDE scheme is specified in the Proposed Learning Algorithm subsection.

Training and Execution Protocol

The learning process follows the standard reinforcement-learning feedback loop under the asynchronous, event-driven bus-control setting and the CTDE scheme described above, illustrated in Figure 3.3. The loop, repeated at every control event during training, proceeds as follows:

1. **Observe.** When bus agent i arrives at a control stop, the Python environment assembles its local observation $s_{i,t}$ (Section 3.2.7) from the analytical bus model and the active stochastic generators.

2. **Act.** The shared policy network selects an action $a_{i,t}$ (holding strength and skip decision) from $s_{i,t}$ using an ϵ-greedy rule over the estimated Q-values during training, and greedily ($\arg\max_a Q$) at evaluation.

3. **Transition and reward.** The environment applies $a_{i,t}$, advances the simulation to the next control event, and returns the event-discounted reward $r_{i,t}$ (Section 3.2.7).

4. **Store.** The transition $(s_{i,t}, a_{i,t}, r_{i,t}, s_{i,t'})$, where t' is the agent's next control event, is written to a shared experience replay buffer common to all agents under parameter sharing.

5. **Update.** Mini-batches sampled from the replay buffer update the shared online network by minimizing the DDQN temporal-difference loss against a slowly-updated target network; experience from every bus updates the single shared parameter set.

This loop is the feedback mechanism that distinguishes the learned controller from the fixed-rule baselines, which compute a holding time directly from the current headways with no update step.

The protocol is split into a training phase and an execution (evaluation) phase:

- **Centralized training phase.** The loop is run for a fixed episode budget with D and T always active while S, W, and B are domain-randomized according to Table 3.1. This follows the domain-randomization approach of Tang et al. [58]. Every bus contributes its own local transition and per-agent reward to the shared replay buffer; experience from all buses updates the one shared network. No shared team reward or joint-state actor input is used.

- **Decentralized execution phase.** The trained network's weights are frozen. During Stage A, Stage B, and ablation evaluation, each bus acts independently and greedily on its local observation $s_{i,t}$, with no centralized action selector and no weight updates. The controller faces held-out stochastic realizations drawn from the declared condition definitions.

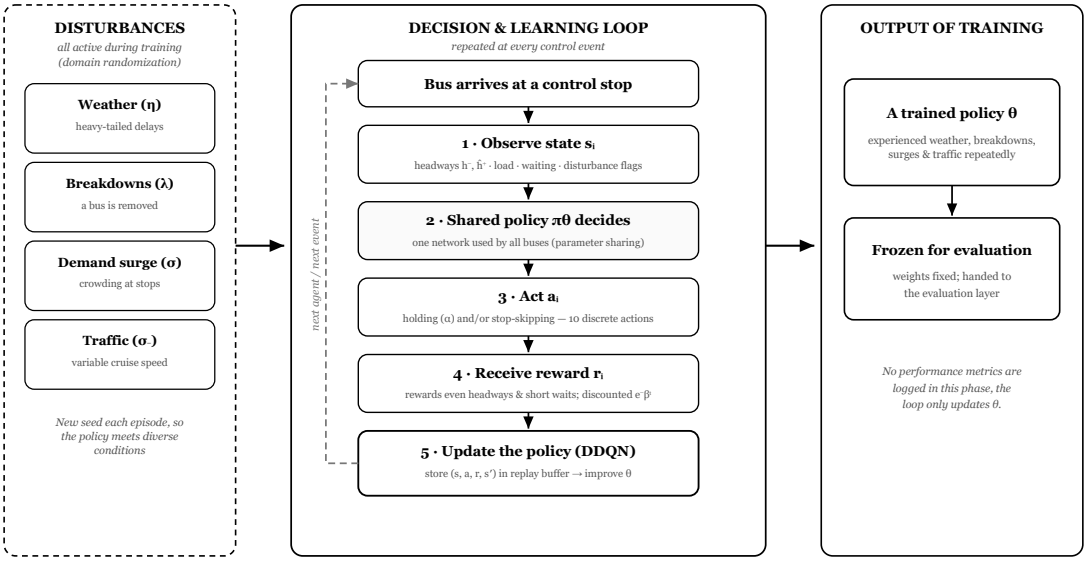

Figure 3.3: The asynchronous agent–environment cycle during training. Authors' illustration, following the agent–environment cycle formalism of Terry et al. [60].

Because heavy-tailed returns can destabilize value-based learning if injected without safeguards [55, 56], training under disturbance is paired with the stabilizers discussed in Section 3.3: DDQN's double estimator, reward clipping, and slow target-network updates. If training under the full disturbance set proves unstable at the highest intensities despite these measures, a curriculum that ramps disturbance intensity over the course of training [58] is retained as a fallback.

3.2.8 Baseline Controllers for Comparison

The MARL policy is benchmarked against three baseline controllers: **No Control (NC)**, **Forward Headway (FH)**, and **Even Headway (EH)**. These are the standard benchmark set in the MARL bus-control literature; both Wang and Sun [30] and Rodriguez et al. [29] use this trio. Using the same baselines makes the results directly comparable to published work.

No Control (NC)

No holding or skipping is applied. Each bus dwells only for the time required to serve boarding and alighting passengers, then departs immediately. NC isolates uncontrolled behavior under each activation-matrix condition. If MARL cannot beat NC, the learned intervention is not operationally justified. Under combined disturbance, NC has no corrective mechanism for the added surge, weather, or breakdown stress.

Forward Headway (FH)

The classical holding rule introduced by Daganzo [22]. When a bus arrives at a control stop, its holding time is:

$$d_{FH} = \max\{0,\, \bar{d} + g\,(H_0 - h^-)\} \tag{3.12}$$

where H_0 is the desired (scheduled) headway, h^- is the forward headway (time gap to the bus ahead), $\bar{d}$ is the average slack delay at equilibrium, and $g > 0$ is a control gain parameter. If a bus is running closer to the bus ahead than scheduled ($h^- < H_0$), it holds for an additional period proportional to the gap deficit; if it is already at or beyond the desired spacing ($h^- \geq H_0$), it does not hold. Parameters $\bar{d}$ and g follow Daganzo's original values. FH represents local, single-agent reactive control: each bus reacts only to information about the bus immediately ahead. Because Eq. (3.12) conditions only on h^-, FH is expected to partially correct demand- and traffic-induced bunching but to respond poorly to a breakdown: it has no observation of the enlarged gap a failed bus leaves for the bus behind it, so the trailing bus's holding decision is uninformed by the breakdown until the irregularity has already propagated back to it.

Even Headway (EH)

A more globally aware rule that holds buses to equalize the forward and backward gaps simultaneously, following Rodriguez et al. [29]:

$$T_{EH} = \max\left\{ \min\left[\frac{\hat{h}^{+} - h^{-}}{2},\ 0.4\,H_0 \right],\ 0 \right\} \tag{3.13}$$

where h^{-} is the forward headway, $\hat{h}^{+}$ is the estimated backward headway (time gap to the bus behind), and $\frac{\hat{h}^{+}-h^{-}}{2}$ is the holding time that would split the difference evenly between the two gaps. The hold is capped at $0.4\,H_0$ (40% of the scheduled headway), the bound Rodriguez et al. used to prevent excessive intervention. This cap matches the maximum value in the MARL action set $\Omega = \{0, 0.1, 0.2, 0.3, 0.4\}$, so EH and MARL are constrained to the same maximum hold and the comparison is fair. EH represents globally-aware reactive control: it uses information about both neighbors but follows a fixed rule rather than a learned policy. Because Eq. (3.13) is a fixed linear rule computed from the currently observed gaps, EH is expected to handle breakdowns better than FH, since the enlarged backward gap $\hat{h}^{+}$ enters the rule directly, but it has no mechanism to anticipate or discount the heavier-tailed delays that the weather generator introduces, since the rule was not designed with a heavy-tailed disturbance in mind.

Baseline Comparisons

The three controllers span the comparison space. NC tests whether intervention helps at all. FH tests whether MARL outperforms classical local reactive control. EH tests whether MARL outperforms classical global reactive control and EH is the stronger of the two classical baselines, consistently reported as the best non-RL controller in the literature [29, 30]. The acceptance criterion for the MARL policy in Stage A (Section 3.2.10) is therefore stated relative to EH: matching or beating EH is the meaningful test.

This three-way comparison also allows the study to attribute MARL's advantage. If MARL beats NC but not FH or EH, the benefit is from intervening, not from learning. If MARL beats FH but not EH, the benefit is from global awareness, not from learning. Only if MARL beats EH does the benefit come from the learned policy itself. The same logic applies in Stage B: the disturbance-handling claim is meaningful only if MARL's advantage over EH grows with η, since both controllers see the same disturbance realizations.

State-of-the-art MARL methods such as CAAC and IQNC-M [30] were considered as additional baselines but excluded. Including them would shift the research question from "does the declared MARL controller remain robust on Route 801?" to "which MARL variant is best?", a different question requiring a larger algorithm-

comparison budget.

3.2.9 Data Analysis Methods

The three response variables logged per run are defined as follows. **Mean passenger waiting time** ($\bar{W}$) is the average time elapsed from a passenger's arrival at a stop to their successful boarding, averaged across all passengers served and all stops over one simulated operating day:

$$\bar{W} = \frac{1}{P} \sum_{p=1}^{P} \left(t_p^{\text{board}} - t_p^{\text{arrive}} \right) \tag{3.14}$$

where P is the total number of passengers served in the run and t_p^{board}, t_p^{arrive} are the boarding and arrival times of passenger p. **Mean total travel time** ($\bar{T}$) is the average elapsed time from a bus's departure from the origin terminal to its arrival at the final stop of the sub-corridor, averaged across all bus trips completed during the simulated day. **Headway coefficient of variation** (CV_h) measures headway regularity:

$$CV_h = \frac{\sigma_h}{\mu_h} \tag{3.15}$$

where σ_h and μ_h are the standard deviation and mean of observed inter-bus headways recorded at all stops over one simulated operating day. $CV_h = 0$ denotes perfectly regular headways; larger values indicate increasing bunching severity. This construction mirrors the baseline coefficient of variation CV_0 already defined for travel time (Table 3.4), applied here to the headway distribution instead.

For each (control strategy, condition) cell, $N_{\text{runs}} \geq 30$ independent Monte Carlo runs are executed using matched random seeds across strategies. Three response variables are logged per run: mean passenger waiting time, mean total travel time, and headway coefficient of variation. The minimum supplies repeated-run variance estimates but does not by itself guarantee normality under heavy tails; distribution checks and bootstrap confidence intervals are therefore required [56, 71].

Statistical significance of performance differences between controllers will be assessed at a significance level of $\alpha = 0.05$. Because the same random seeds are reused across controllers within a disturbance level, comparisons between two controllers at the same disturbance level are naturally paired: the only difference between two runs is the controller, not the disturbance realization. Paired-sample tests will therefore be used for within-disturbance-level comparisons; unpaired tests will be used where seed matching does not apply (for example, comparing the same controller across different disturbance levels). Multiple-comparison correction will be applied across the four controllers to control the family-wise error rate.

For every disturbed condition, performance degradation is computed relative to

the same controller's D+T Stage A result. Confidence intervals on the degradation ratio are computed by bootstrap resampling because ratios of means can be skewed under heavy-tailed stress [72]. Hypothesis tests, normality diagnostics, family-wise correction, bootstrap replicate count, and random seeds are fixed in the analysis configuration before final evaluation, following reinforcement-learning evaluation guidance [55, 56]. Run logs and analysis code are version-controlled; large generated outputs are checksum-recorded but not committed.

3.2.10 Evaluation Methods

Stage A: baseline evaluation. The frozen MARL policy and three baselines are evaluated under D+T, with S/W/B off, for $N_{\text{runs}} \geq 30$ matched seeds. This is a stochastic empirical baseline rather than an "ideal" deterministic corridor. Parity with EH on waiting time and improvement over NC on headway variation remain the minimum acceptance targets.

Stage A acceptance criterion

(i) Mean passenger waiting time no worse than EH (no statistically significant degradation at $p < 0.05$ with multiple-comparison correction), and ideally a statistically significant improvement.

(ii) A statistically significant reduction in headway coefficient of variation relative to NC.

Figure 3.4 shows the format in which these Stage A comparisons are reported across the four controllers and three response variables.

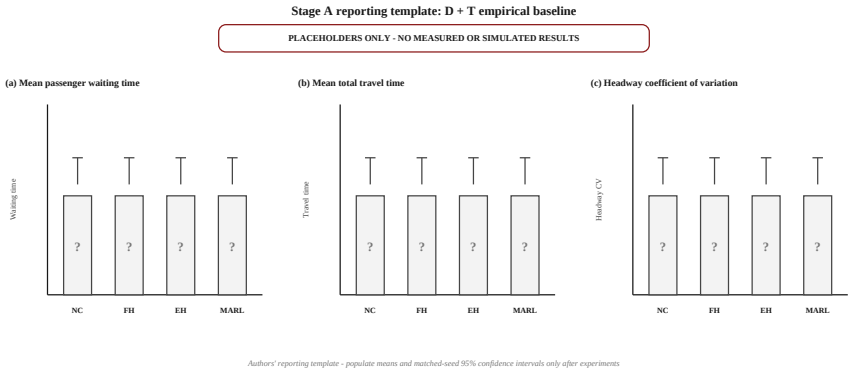

Figure 3.4: Illustrative output of the Stage A D+T baseline evaluation, comparing the four controllers (NC, FH, EH, MARL) across the three response variables: (a) mean passenger waiting time, (b) mean total travel time, and (c) headway coefficient of variation. Error bars represent 95% confidence intervals over $N \geq 30$ Monte Carlo runs with matched seeds. Numerical values shown are illustrative and do not reflect measured results. Authors' illustration.

Stage B: combined-disturbance evaluation. Controllers are evaluated under D+T+S+W+B. One series uses the observed ordinary-rain exposure/multiplier;

a separate extrapolative series sweeps synthetic $\eta \in \{0.3, 0.6, 1.0, 1.3\}$. Each controller/condition cell has $N_{\text{runs}} \geq 30$ matched seeds. Outputs include response means and intervals, degradation relative to the same controller's Stage A result, and MARL's paired advantage over the strongest baseline. Single-disturbance ablations evaluate D+T+S, D+T+W (observed ordinary-rain definition), and D+T+B. Synthetic η results are never relabeled as observed weather categories.

Stage B acceptance criterion The MARL policy is reported as performing well under disturbance if its mean waiting time stays below that of the best-performing baseline across the full sweep, with statistical significance under the same correction as Stage A. Figure 3.5 illustrates the matched-seed Monte Carlo evaluation procedure used for these cells.

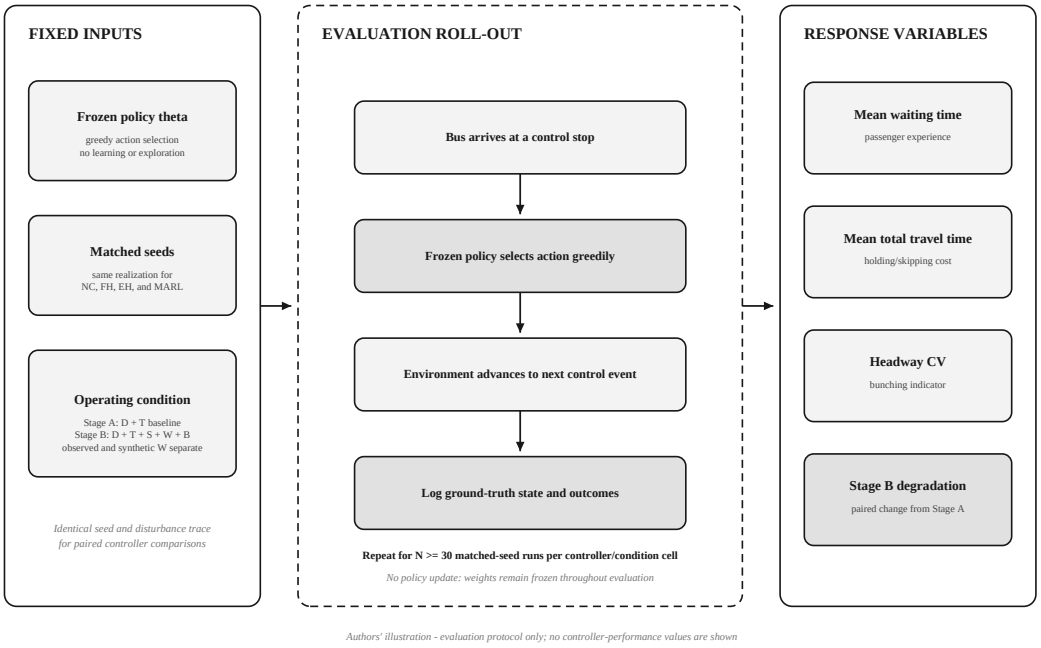

Figure 3.5: The Monte Carlo evaluation layer. The frozen policy θ selects the highest-valued action at each control stop without exploration or weight updates. Matched seeds ensure that every controller faces the same disturbance realization, so performance differences reflect the control strategy alone. Three response variables are recorded per run: mean passenger waiting time, mean total travel time, and headway coefficient of variation. Authors' illustration.

3.3 Methodological Challenges

Three challenges shape the experimental design. The design is scoped to remain feasible on accessible hardware (a single cloud-notebook GPU runtime), and a tiered scope plan (stated at the end of this section) guarantees a defensible minimum deliverable even if the full sweep cannot be completed.

Q-value overestimation under heavy-tailed returns. Value-based methods such as DQN overestimate Q-values [69], and this bias grows when returns are heavy-tailed, as they are under high-intensity disturbance [55, 56]. The DDQN double estimator addresses this by decoupling action selection from evaluation [69]. Because disturbances are active during training, reward clipping and target-network soft updates are used as additional stabilizers throughout, not only at the highest intensities.

Computational cost. At the minimum $N_{\text{runs}} = 30$, Stage A requires $4 \times 30 = 120$ runs. The observed-weather combined cell adds 120; the four-level synthetic combined sweep adds $4 \times 4 \times 30 = 480$; and the three single-disturbance ablations add $3 \times 4 \times 30 = 360$. The declared minimum is therefore 1,080 evaluation runs. SUMO is restricted to calibration, while event-driven evaluation is parallelized across seeds. Any reduced tier must preserve Stage A, the observed-weather combined cell, and at least one synthetic out-of-support level, and must be reported as a scope reduction rather than silently changing the matrix.

Timeline feasibility. The declared minimum evaluation suite contains 1,080 runs, as calculated above. No wall-clock duration is claimed before the Route 801 event-driven simulator is profiled: representative Stage A, observed-weather, synthetic-weather, and breakdown episodes will be benchmarked first, and the measured per-run CPU time, memory use, and training throughput will determine the parallelization and hardware plan. Rodriguez et al. [29] report a two-phase 500+300-episode training design, but that external episode count is a sizing reference rather than evidence of this implementation's runtime. If the measured cost requires a reduced tier, it must preserve Stage A, the observed-weather combined cell, at least one synthetic out-of-support level, and the declared matched-seed comparison; every omitted cell must be reported as a scope reduction rather than silently changing the activation matrix.

Bibliography

[1] R. Cervero, *Informal Transport in the Developing World*. Nairobi, Kenya: UN-Habitat, 2000.

[2] United Nations Department of Economic and Social Affairs, Population Division, "World urbanization prospects: The 2018 revision," Tech. Rep. ST/ESA/SER.A/420, United Nations, New York, NY, USA, 2019.

[3] S. R. D. Castro, C. A. C. Tadle, and B. G. Carcellar III, "Spatiotemporal analysis of traffic accidents and traffic congestion along commonwealth avenue using crowdsourced waze traffic data," *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. X-5/W4-2025, pp. 165–172, 2026.

[4] S. F. Chua, *Stress Among Bus and Personal Drivers in the Philippines*. PhD thesis, Queensland University of Technology, Brisbane, QLD, Australia, 2026.

[5] Philippine Star (Department of Transportation data), "EDSA busway ridership hits over 66 million in 2025." Philstar.com news report citing Department of Transportation (DOTr) records, Jan. 2026. Reports 2025 annual ridership of 66.67 million (up from 63.02 million in 2024), average daily ridership of approximately 182,655, and a single-day peak of 321,186 passengers in April 2025.

[6] S. M. Gaspay, N. C. Tiglao, M. A. Tacderas, N. J. Tolentino, and A. C. Ng, "Reforms in metro manila's bus transport system hastened by the covid-19 pandemic: A policy capacity analysis of the edsa busway," *Research in Transportation Economics*, vol. 100, p. 101305, 2023.

[7] L. J. Ollero, K. Vergel, and N. C. C. Tiglao, "A microsimulation model of an exclusive bus lane: The case of edsa busway," *Philippine Transportation Journal*, vol. 7, no. 2, 2024.

[8] N. C. C. Tiglao, E. Sanciangco, and N. J. Tolentino, "Digitally enabled collaborative governance for sustaining bus reforms on the edsa busway in metro manila," *Research in Transportation Economics*, vol. 113, p. 101633, 2025.

[9] J. M. B. Espino, R. G. L. Larraquel, A. S. R. L. Purisima, A. M. B. Valenzuela, and M. N. Borromeo, "Impacts of different rainfall intensities on key traffic flow parameters at north luzon expressway using underwood's exponential model," in *Proceedings of the 25th Annual Conference of the Transportation Science Society of the Philippines*, 2018.

[10] J. Mangaluz, "Flooding affects bus trips; train ops continue," *The Philippine Star*, July 2024.

[11] Philippine Information Agency, "Edsa bus carousel to undergo emergency road repair." PIA News, Aug. 2023.

[12] S. Locus, "Dpwh to close portions of edsa bus carousel for emergency repairs," 2023.

[13] L. Paget-Seekins and M. Tironi, "The publicness of public transport: The changing nature of public transport in latin american cities," *Transport Policy*, vol. 49, pp. 176–183, 2016.

[14] J. E. Barrera Hernandez, L. E. Tarazona Torres, A. Tabares, and D. Álvarez-Martínez, "Optimization of bus dispatching in public transportation through a heuristic approach based on passenger demand forecasting," *Smart Cities*, vol. 8, no. 3, 2025.

[15] A. Ceder, *Public Transit Planning and Operation: Theory, Modeling and Practice*. Oxford, UK: Butterworth-Heinemann, 1st ed., 2007.

[16] Y. Wang, D. Zhang, L. Hu, Y. Yang, and L. H. Lee, "A data-driven and optimal bus scheduling model with time-dependent traffic and demand," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 9, pp. 2443–2452, 2017.

[17] T. Chen, X. Chen, and Y. Wang, "Dynamic bus dispatching via deep reinforcement learning: A survey," *arXiv preprint arXiv:2401.12455*, 2024.

[18] J. Wang and L. Sun, "Dynamic holding control to avoid bus bunching: A multi-agent deep reinforcement learning framework," *Transportation Research Part C: Emerging Technologies*, vol. 116, p. 102661, 2020.

[19] Capital Metropolitan Transportation Authority, "CapMetro Rapid," 2026. Current service information; accessed 2026-08-23 and not used to label 2021 APC direction codes.

[20] Capital Metropolitan Transportation Authority, "APC Raw July 2021–December 2021." Texas Open Data Portal, Socrata dataset im6q-3pc9, 2021. Accessed 2026-08-23.

[21] D. Kantor, N. W. Casey, M. J. Menne, and A. Buddenberg, "Local Climatological Data, Version 2," 2023. Austin Camp Mabry USW00013958 and Austin-Bergstrom USW00013904, July–December 2021 subset; accessed 2026-08-23.

[22] C. F. Daganzo, "A headway-based approach to eliminate bus bunching: Systematic analysis and comparisons," *Transportation Research Part B: Methodological*, vol. 43, no. 10, pp. 913–921, 2009.

[23] Y. Huang *et al.*, "Bus arrival time prediction using machine learning techniques in uncertain environments," *IEEE Access*, vol. 7, pp. 93224–93235, 2019.

[24] S. Patil *et al.*, "Advanced predictive analytics for public transit control and fleet scheduling," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 2, pp. 11176–11186, 2025.

[25] A. Usman, T. M. Ali, and C. Irawan, "Machine learning approaches for real-time traffic density estimation and public transport optimization," *Engineering Proceedings*, vol. 107, no. 1, 2025.

[26] T. Alexandre, F. Bernardini, J. Viterbo, and C. E. Pantoja, "Machine learning applied to public transportation by bus: A systematic literature review," *Transportation Research Record*, vol. 2677, no. 7, pp. 639–660, 2023.

[27] J. Pang *et al.*, "A predictive holding control strategy for bus transit reliability using real-time information," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 10, pp. 3874–3884, 2019.

[28] A. Momenikorbekandi and M. Abbod, "Intelligent scheduling based on reinforcement learning approaches: Applying advanced q-learning and sarsa reinforcement learning models for the optimisation of job shop scheduling problems," *Electronics*, vol. 12, no. 23, 2023.

[29] J. Rodriguez, H. N. Koutsopoulos, S. Wang, and J. Zhao, "Cooperative bus holding and stop-skipping: A deep reinforcement learning framework," *Transportation Research Part C: Emerging Technologies*, vol. 155, p. 104308, 2023.

[30] J. Wang and L. Sun, "Robust dynamic bus control: A distributional multi-agent reinforcement learning approach," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 4, pp. 4075–4088, 2023.

[31] Y. Zhao, D. A. Ramli, and M. C. Lim, "Dynamic bus holding control using spatial-temporal data: A deep reinforcement learning approach," in *AI 2022: Advances in Artificial Intelligence, LNAI 13728*, pp. 586–598, Springer, 2022.

[32] Y. Zhang and L. Zheng, "Single agent robust deep reinforcement learning for bus fleet control," 2025.

[33] J. Wang and L. Sun, "Multi-objective multi-agent deep reinforcement learning to reduce bus bunching for multiline services with a shared corridor," *Transportation Research Part C: Emerging Technologies*, vol. 155, p. 104309, 2023.

[34] F. Christianos, G. Papoudakis, A. Rahman, and S. V. Albrecht, "Scaling multi-agent reinforcement learning with selective parameter sharing," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, vol. 139 of *Proceedings of Machine Learning Research*, pp. 1989–1998, PMLR, 2021.

[35] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022.

[36] B. Sun, Y. Yang, L. Dong, H. Lu, and Z. Wang, "Analysis and dynamic prediction of bus dwell time under rainfall conditions," *Promet – Traffic&Transportation*, vol. 37, no. 1, pp. 105–121, 2025.

[37] M. Patil, Q. Ahmed, and S. Midlam-Mohler, "Travel time and weather-aware traffic forecasting in a conformal graph neural network framework," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 12, pp. 21734–21744, 2025.

[38] J. K. Gupta, M. Egorov, and M. J. Kochenderfer, "Cooperative multi-agent control using deep reinforcement learning," in *Autonomous Agents and Multiagent Systems (AAMAS 2017 Workshops), LNCS 10642*, pp. 66–83, Springer, 2017.

[39] L. Buşoniu, R. Babuška, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 2, pp. 156–172, 2008.

[40] Z. Li, K. Cai, and P. Zhao, "Departure scheduling for multi-airport system using multi-agent reinforcement learning," in *2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)*, pp. 1–9, 2023.

[41] W. Sun, Y. Zou, N. Guan, X. Zhang, G. Du, and Y. Wen, "Graph attention network–based deep reinforcement learning scheduling framework for in-vehicle time-sensitive networking," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 7, pp. 9825–9836, 2024.

[42] T. Zuo, H. Liu, S. Yang, W. Wang, Y. Peng, and R. Wang, "A reinforcement learning method for automated guided vehicle dispatching and path planning considering charging and path conflicts at an automated container terminal," *Journal of Marine Science and Engineering*, vol. 14, no. 1, 2026.

[43] X. Huang, Y. Tian, J. Li, N. Zhang, X. Dong, Y. Lv, and Z. Li, "Joint autonomous decision-making of conflict resolution and aircraft scheduling based on triple-aspect improved multi-agent reinforcement learning," *Expert Systems with Applications*, vol. 275, p. 127024, 2025.

[44] R. Bokade, X. Jin, and C. Amato, "Multi-agent reinforcement learning based on representational communication for large-scale traffic signal control," *IEEE Access*, vol. 11, pp. 47646–47658, 2023.

[45] Y. Cao, Z. Xu, and M. Meng, "Train rescheduling method based on multi-agent reinforcement learning," in *2022 IEEE 6th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, pp. 301–305, 2022.

[46] F. Wang, P. Ding, Y. Bi, and J. Qiu, "A multi-agent deep reinforcement learning approach for multiple agvs scheduling in automated container terminals," in *2024 China Automation Congress (CAC)*, pp. 3564–3569, 2024.

[47] L. Nie, D. Qi, B. Liu, P. Li, H. Bao, and H. He, "Cmrm: Collaborative multi-agent reinforcement learning for multi-objective traffic signal control," *IEEE Transactions on Consumer Electronics*, vol. 71, no. 2, pp. 2793–2805, 2025.

[48] Q. Xu, Z. Zhang, J. Li, and X. Qi, "Hierarchical multi-agent reinforcement learning algorithm for multi-uav roundup strategy," *Applied Mathematical Modelling*, vol. 155, p. 116728, 2026.

[49] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[50] W. Chen, K. Zhou, and C. Chen, "Real-time bus holding control on a transit corridor based on multi-agent reinforcement learning," in *Proceedings of the 19th IEEE International Conference on Intelligent Transportation Systems (ITSC)*, (Rio de Janeiro, Brazil), pp. 100–106, 2016.

[51] K. Katzilieris, E. Kampitakis, and E. I. Vlahogianni, "A multi-agent reinforcement learning framework for integrated traffic signal control and dynamic bus lane access management," *Transportation Research Part C: Emerging Technologies*, vol. 182, p. 105391, 2026.

[52] D. Liu, F. Xiao, J. Luo, and F. Yang, "Deep reinforcement learning-based holding control for bus bunching under stochastic travel time and demand," *Sustainability*, vol. 15, no. 14, p. 10947, 2023.

[53] H. Shi, Q. Nie, S. Fu, X. Wang, Y. Zhou, and B. Ran, "A distributed deep reinforcement learning-based integrated dynamic bus control system in a connected

environment," *Computer-Aided Civil and Infrastructure Engineering*, vol. 37, no. 15, pp. 2016–2032, 2022.

[54] P. C. Guedes and D. Borenstein, "Real-time multi-depot vehicle type rescheduling problem," *Transportation Research Part B: Methodological*, vol. 108, pp. 217–234, 2018.

[55] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[56] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.

[57] A. Che, Z. Wang, and C. Zhou, "Multi-agent deep reinforcement learning for recharging-considered vehicle scheduling problem in container terminals," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 11, pp. 16855–16868, 2024.

[58] Y. Tang, A. Qu, X. Jiang, B. Mo, S. Cao, J. Rodriguez, H. N. Koutsopoulos, C. Wu, and J. Zhao, "Robust reinforcement learning strategies with evolving curriculum for efficient bus operations in smart cities," *Smart Cities*, vol. 7, no. 6, pp. 3658–3677, 2024.

[59] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using SUMO," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pp. 2575–2582, IEEE, 2018.

[60] J. K. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. Santos, R. Perez, C. Horsch, C. Dieffendahl, N. L. Williams, Y. Lokesh, and P. Ravi, "PettingZoo: Gym for multi-agent reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.

[61] M. Towers, A. Kwiatkowski, J. K. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[62] Capital Metropolitan Transportation Authority, "CapMetro GTFS." Texas Open Data Portal, Socrata asset r4v4-vz24, 2026. Current feed; accessed 2026-08-23 and not substituted for a 2021 snapshot.

[63] UK Highways Agency, "Design manual for roads and bridges, volume 12, section 2: Traffic appraisal in urban areas," tech. rep., The Stationery Office, London, 1996.

[64] Federal Highway Administration, "Traffic analysis toolbox volume iii: Guidelines for applying traffic microsimulation modeling software," Tech. Rep. FHWA-HOP-18-036, U.S. Department of Transportation, 2019.

[65] J. F. C. Kingman, *Poisson Processes*. Oxford University Press, 1993.

[66] R. Bellman, "A Markovian decision process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957.

[67] S. J. Bradtke and M. O. Duff, "Reinforcement learning methods for continuous-time Markov decision problems," in *Advances in Neural Information Processing Systems*, vol. 7, pp. 393–400, 1995.

[68] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Proceedings of the 11th International Conference on Machine Learning (ICML)*, pp. 157–163, Morgan Kaufmann, 1994.

[69] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pp. 2094–2100, 2016.

[70] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[71] L. Wasserman, *All of Statistics: A Concise Course in Statistical Inference*. New York: Springer, 2004.

[72] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall/CRC, 1993.

University of Santo Tomas
Faculty of Engineering
Electronics Engineering Department

Research Workplan - Calendar of Activities (June 2026 to April 2027)

Research Title: An Evaluation of Multi-Agent Reinforcement Learning for Dynamic Bus Scheduling Under Non-Ideal Conditions

Proponents: Badal, Lopez, Mananguit, Medenilla, Marquez Adviser: Asst. Prof. Kanny Krizzy D. Serrano, MSc

Research Group: B3 **UG Cohort:** 2026

Objectives	Expected Outcome	Milestone Indicators / Measure	Work Components and/or Tasks	Jun 26				Jul 26				Aug 26				Sep 26				Oct 26				Nov 26				Dec 26				Jan 27				Feb 27				Mar 27				Apr 27				Risk Identification
				1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	
	Preliminary Preparation	Pre Processed Traffic Dataset	Request traffic dataset																																										Delay in dataset collection
			Familiarize Software Tools																																									
			Learn MARL Implementation																																									
SO 1: To construct a stochastic traffic simulation environment representing a defined traffic area.	EO 1.1: A simulation model of a defined operational scope, validated against empirical traffic data to establish an environment that accurately approximates real-world traffic.	GEH & Root Mean Square Error (RMSE)	Map the road network and stations in SUMO.																																								
			Import Processed Traffic Data																																								
			Compare simulated travel times vs. real-world data.																																								
			Parameter Calibration																																									calibration failure/error
			Documentation																																								
	EO 1.2: An environment simulator capable of introducing environmental variability through adjustable parameters to simulate non-ideal operating conditions.	Coefficient of Variation (CoV) and Scaling factors from data sets	Program Python script for passenger demand multipliers via TraCI.																																							
			Program Python script for weather-induced speed limiters																																								
			Measure baseline variance/chaos.																																								
			Document generator limits.																																								
SO 2: To develop and implement a Multi-Agent Reinforcement Learning (MARL) approach for traffic scheduling.	EO 2.1: A formulated agent architecture defining individual bus units with discrete state and action space with a continuous reward function that punishes irregularities and passenger service delays.	Reward Convergence Curve	Define state and action spaces for bus agents.																																							
			Implement MARL architecture																																								
			Integrate MARL code with SUMO environment.																																								
			Formulate mathematical reward function																																								Non-convergence of the reward policy
			Set up training environment																																								Hardware Constraints or Google Colab Limitations
			Execute training loop for target episodes																																							Testing small scale simulations might take time. Training in general will take a long time
			Analyze reward convergence graphs.																																							
			Save and extract trained model weights.																																								
SO 3: To evaluate the scheduling performance of the MARL approach under both ideal operating conditions and non-ideal conditions.	EO 3.1:A quantitative and statistical assessment of MARL scheduling performance under ideal conditions, measuring covering passenger waiting time, travel time, and headway coefficient of variation, tested against baseline methods.	Average Passenger Waiting Tme, Headway Variance CV, Travel Time	Create Python Script for, No Control (NC), Forward Headway (FH), Even Headway (EH) control strategies																																							Baseline Control Calibration Inaccuracy
			Execute No Control (NC), Forward Headway (FH), Even Headway (EH), for ideal conditions																																							
			Execute trained MARL simulation without disturbances.																																							
			Extract system-wide parameters for all controls.																																							
			Documentation																																							
	EO 3.2: A quantitative and statistical assessment of MARL scheduling performance under ideal conditions, measuring covering passenger waiting time, travel time, and headway coefficient of variation, tested against baseline methods.	Average Passenger Waiting Tme, Headway Variance CV, Travel Time, Degradation	Execute No Control (NC), Forward Headway (FH), Even Headway (EH), for non-ideal conditions																																							
			Extract system-wide parameters for all controls.																																							
			Comparison against ideal conditions																																							

Legend:

Phase 1

Phase 2

Phase 3